

# Neural Proofs for Sound Verification of Complex Programs and Systems

Alessandro Abate

OXCAV - Department of Computer Science - University of Oxford

TASE - 6 July 2026



*[references at end of deck]*

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

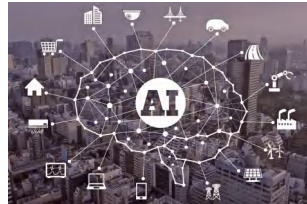
# Cyber-Physical Systems: from code to control

- complex embedded systems
  - interleaving of cyber/digital components within physical/analogue dynamics
  - computation, dynamics & control
  - safety-critical applications
- sound and automated verification
- correct-by-design control synthesis



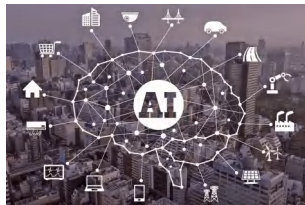
# Cyber-Physical Systems: from code to control

- complex embedded systems
  - interleaving of cyber/digital components within physical/analogue dynamics
  - computation, dynamics & control
  - safety-critical applications
- sound and automated verification
- correct-by-design control synthesis



[source: embedded.com]

- **AI-enhanced embedded intelligence**
  - interleaving of **AI +** cyber/digital components within physical/analogue dynamics
  - **learning +** computation, dynamics & control
  - safety-critical applications
- ?? **sound** and automated verification
- ?? **correct-by-design** control synthesis
- **how might AI-enhanced models** be verified / safely designed?



[source: embedded.com]

# Models for dynamics and computation - *faraway, so close!*

- dynamical **models** for control

$$x \in \mathbb{R}^n, \quad x^+ = \begin{cases} f_1(x, a, w), & x \geq 0 \\ f_2(x, a, w), & x < 0 \end{cases}$$

- uncertainty

- **non-deterministic** or **probabilistic/stochastic**
- epistemic vs aleatoric
- endogenous vs exogenous

- safety,  
reachability,  
stability

- invariants & barrier certificates,  
reach-set computation,  
Lyapunov functions

- **models** of software programs

```
int x, a, w;  
...  
x = rand(1,...,10);  
a = input(1,...,3);  
while (x >= 0) {  
  w = Bernoulli(0.5);  
  if (a == 1 && w) { x = 2 * x; }  
  else if (a == 1 && !w) { a++; }  
  else { x = x - w; }  
}  
...  
if (x < 0) { ...
```

- invariance,  
termination,  
assertion check/violation
- program and loop invariants,  
reachability over control flow graph  
ranking functions, progress measures

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

# Formalising dynamical behaviour

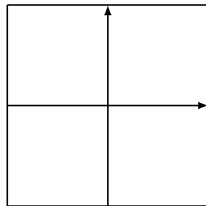


- as requirements, specifications (e.g., safety) for verification
- as precise tasks, objectives (e.g., reachability) for control design/synthesis
- without manual reward/cost engineering
- useful for general sequential decision making problems (e.g., controller synthesis, and model-free RL)

# Formalising dynamical behaviour

$$x \in \mathbb{R}^n$$

$$x^+ = f(x)$$

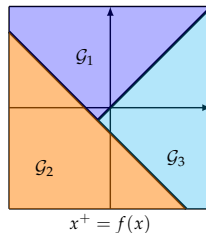
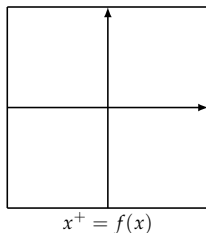


$$x^+ = f(x)$$

$$x \in \mathbb{R}^n$$

$$\mathcal{G}_i \subset \mathbb{R}^n, i \in \{1, \dots, m\}$$

$$x^+ = f(x)$$



|          |                                 |                                 |                                 |         |
|----------|---------------------------------|---------------------------------|---------------------------------|---------|
| $x :$    | $x_0 \rightarrow$               | $x_1 \rightarrow$               | $x_2 \rightarrow$               | $\dots$ |
| $\rho :$ | $\mathcal{G}_{x_0} \rightarrow$ | $\mathcal{G}_{x_1} \rightarrow$ | $\mathcal{G}_{x_2} \rightarrow$ | $\dots$ |

- model's dynamics (behaviour, *cf.* Jan Willems)  $\leftrightarrow$  formal language
- *cf.* supervisory control by constraining dynamics/behaviour

# Formalising dynamical behaviour

- consider class of control problems

$$\forall x_0 \in \mathcal{I},$$

$$\exists T \in \mathbb{N},$$

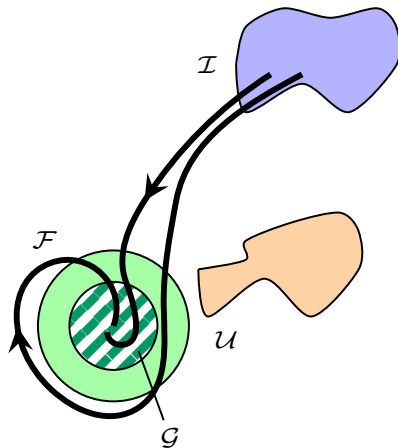
$$\forall k \in \{0, 1, \dots, T-1\},$$

$$\forall \tau \geq T:$$

$$x_T \in \mathcal{G},$$

$$x_k \notin \mathcal{U},$$

$$x_\tau \in \mathcal{F}$$



# Formalising dynamical behaviour

- consider class of control problems

$$\forall x_0 \in \mathcal{I},$$

$$\exists T \in \mathbb{N},$$

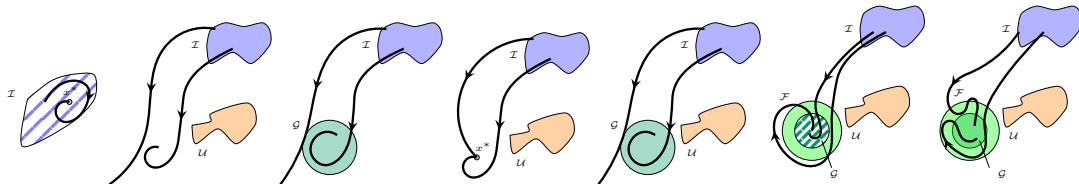
$$\forall k \in \{0, 1, \dots, T-1\},$$

$$\forall \tau \geq T:$$

$$x_T \in \mathcal{G},$$

$$x_k \notin \mathcal{U},$$

$$x_\tau \in \mathcal{F}$$



- class encompasses stability, invariance/safety, reachability, reach-avoid, ... (+ combinations)
- more formally, class can be framed as
  - regular expressions
  - safe/co-safe fragment of linear temporal logic (LTL)
  - DFA

# Formalising dynamical behaviour

- consider class of control problems

$$\forall x_0 \in \mathcal{I},$$

$$\exists T \in \mathbb{N},$$

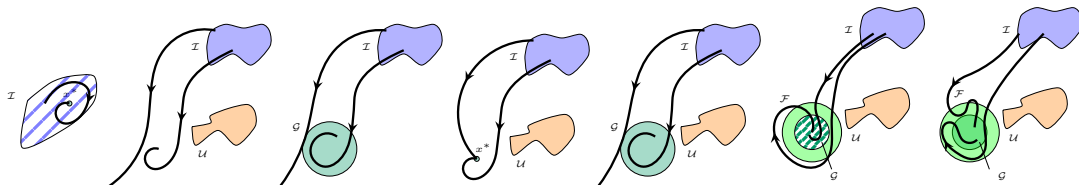
$$\forall k \in \{0, 1, \dots, T-1\},$$

$$\forall \tau \geq T:$$

$$x_T \in \mathcal{G},$$

$$x_k \notin \mathcal{U},$$

$$x_\tau \in \mathcal{F}$$



- connections to:
  - formal languages (reg. exp's)
  - temporal logic (LTL)
  - automata theory (DFA)

# Formalising dynamical behaviour

- consider class of control problems

$$\forall x_0 \in \mathcal{I},$$

$$\exists T \in \mathbb{N},$$

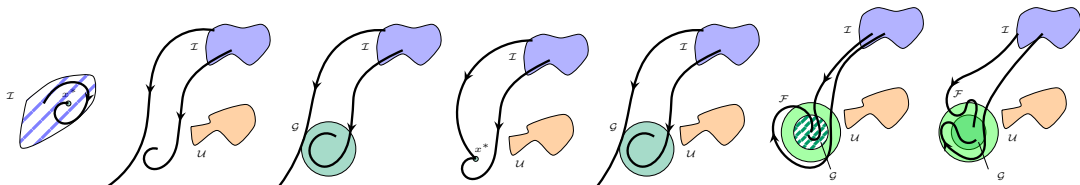
$$\forall k \in \{0, 1, \dots, T-1\},$$

$$\forall \tau \geq T:$$

$$x_T \in \mathcal{G},$$

$$x_k \notin \mathcal{U},$$

$$x_\tau \in \mathcal{F}$$



- connections to:

- formal languages (reg. exp's)
- temporal logic (LTL)
- automata theory (DFA)

$$\exists T \in \mathbb{N}, x_T \in \mathcal{G}$$

$$\equiv$$

$$\Diamond \mathcal{G} \quad \text{FG}$$

# Formalising dynamical behaviour

- consider class of control problems

$$\forall x_0 \in \mathcal{I},$$

$$\exists T \in \mathbb{N},$$

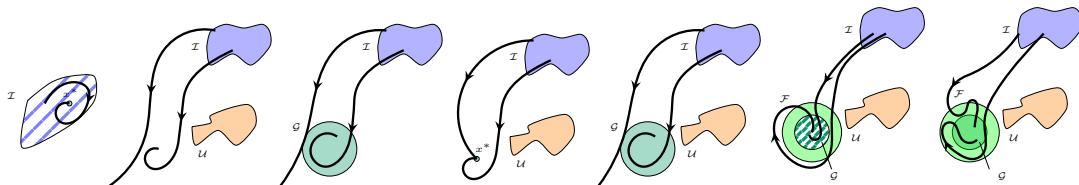
$$\forall k \in \{0, 1, \dots, T-1\},$$

$$\forall \tau \geq T:$$

$$x_T \in \mathcal{G},$$

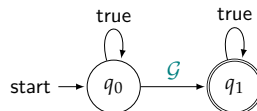
$$x_k \notin \mathcal{U},$$

$$x_\tau \in \mathcal{F}$$

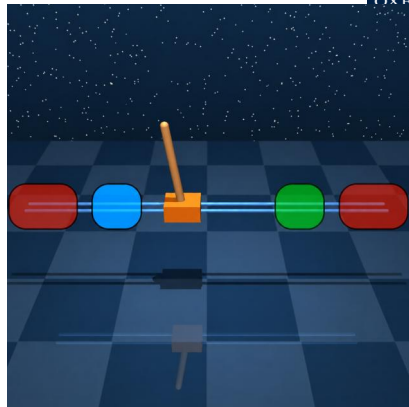
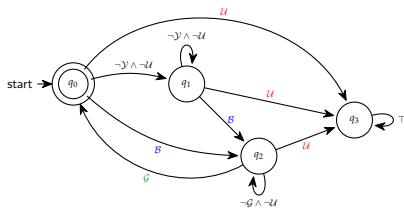


- connections to:

- formal languages (reg. exp's)
- temporal logic (LTL)
- automata theory (DFA)

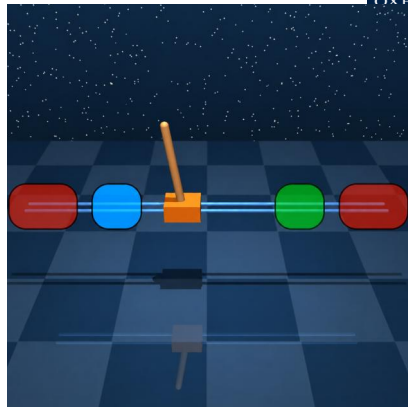
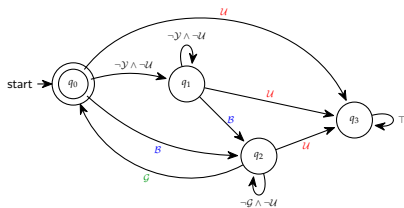


$$GF\mathcal{B} \wedge GF\mathcal{G} \wedge G\neg\mathcal{U}$$



- $\infty$ -horizon specifications
- beyond obligation, to full reactivity in Pnueli-Manna's hierarchy ( $\omega$ -regular languages)
- automata on infinite strings - Büchi (above), Streett (next), etc.

$$GF\mathcal{B} \wedge GF\mathcal{G} \wedge G\neg\mathcal{U}$$



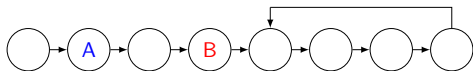
- $\infty$ -horizon specifications
- beyond obligation, to full reactivity in Pnueli-Manna's hierarchy ( $\omega$ -regular languages)
- automata on infinite strings - Büchi (above), Streett (next), etc.

*with rewards? thanks, but no thanks*

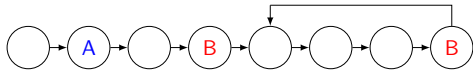
# Deterministic Street Automata (DSA)

- states  $Q$ , alphabet  $\Sigma$ , transition function  $\delta: Q \times \Sigma \rightarrow Q$
- accepting pair  $(A, B)$  of sets of states,  $A, B \subseteq Q$ , and acceptance condition:

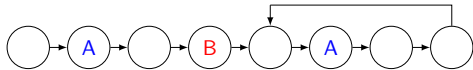
$$\text{GF}(A) \implies \text{GF}(B)$$



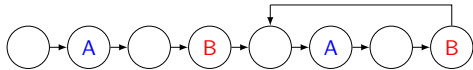
finite A's  
finite B's



finite A's  
infinite B's



infinite A's  
finite B's



infinite A's  
infinite B's

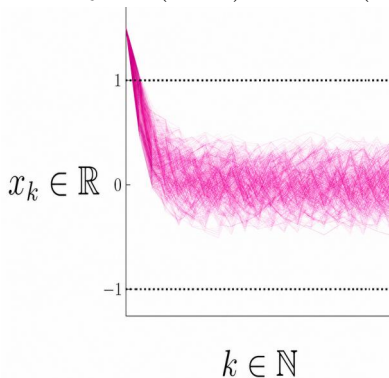
# Deterministic Streett Automata (DSA)

- states  $Q$ , alphabet  $\Sigma$ , transition function  $\delta: Q \times \Sigma \rightarrow Q$
- accepting pair  $(A, B)$  of sets of states,  $A, B \subseteq Q$ , and acceptance condition:

$$\text{GF}(A) \implies \text{GF}(B)$$

$$x^+ = 0.5 \cdot x + w$$

$$x_0 \sim \mathcal{U}(1, 100), \quad w \sim \mathcal{U}(-0.1, 0.1)$$



# Deterministic Streett Automata (DSA)

- states  $Q$ , alphabet  $\Sigma$ , transition function  $\delta: Q \times \Sigma \rightarrow Q$
- accepting pair  $(A, B)$  of sets of states,  $A, B \subseteq Q$ , and acceptance condition:

$$\text{GF}(A) \implies \text{GF}(B)$$

$$x^+ = 0.5 \cdot x + w$$

$$x_0 \sim \mathcal{U}(1, 100), \quad w \sim \mathcal{U}(-0.1, 0.1)$$



select atomic propositions:

$$x < -1, \quad -1 \leq x \leq 1, \quad x > 1$$

map trajectories  $\rightarrow$  traces:

$$\begin{array}{ccccccc} 1.5, & 0.72, & 1.23, & -1.36, & \dots \\ x > 1, & -1 \leq x \leq 1, & x > 1, & x < -1, & \dots \end{array}$$

# Deterministic Streott Automata (DSA)

- states  $Q$ , alphabet  $\Sigma$ , transition function  $\delta: Q \times \Sigma \rightarrow Q$
- accepting pair  $(A, B)$  of sets of states,  $A, B \subseteq Q$ , and acceptance condition:

$$\text{GF}(A) \implies \text{GF}(B)$$

$$x^+ = 0.5 \cdot x + w$$

$$x_0 \sim \mathcal{U}(1, 100), \quad w \sim \mathcal{U}(-0.1, 0.1)$$



select atomic propositions:

$$x < -1, \quad -1 \leq x \leq 1, \quad x > 1$$

map trajectories  $\rightarrow$  traces:

$$\begin{array}{ccccccc} 1.5, & 0.72, & 1.23, & -1.36, & \dots \\ x > 1, & -1 \leq x \leq 1, & x > 1, & x < -1, & \dots \end{array}$$

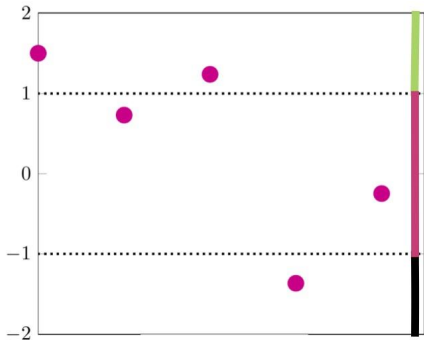
# Deterministic Streott Automata (DSA)

- states  $Q$ , alphabet  $\Sigma$ , transition function  $\delta: Q \times \Sigma \rightarrow Q$
- accepting pair  $(A, B)$  of sets of states,  $A, B \subseteq Q$ , and acceptance condition:

$$\text{GF}(A) \implies \text{GF}(B)$$

$$x^+ = 0.5 \cdot x + w$$

$$x_0 \sim \mathcal{U}(1, 100), \quad w \sim \mathcal{U}(-0.1, 0.1)$$



$$\neg(x < -1) \text{UG}(-1 \leq x \leq 1)$$

select atomic propositions:

$$x < -1, \quad -1 \leq x \leq 1, \quad x > 1$$

map trajectories  $\rightarrow$  traces:

$$\begin{array}{ccccccc} 1.5, & 0.72, & 1.23, & -1.36, & \dots \\ x > 1, & -1 \leq x \leq 1, & x > 1, & x < -1, & \dots \end{array}$$

# Deterministic Streett Automata (DSA)

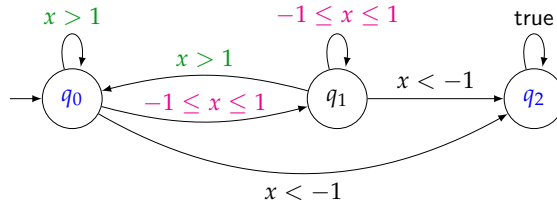
- states  $Q$ , alphabet  $\Sigma$ , transition function  $\delta: Q \times \Sigma \rightarrow Q$
- accepting pair  $(A, B)$  of sets of states,  $A, B \subseteq Q$ , and acceptance condition:

$$\text{GF}(A) \implies \text{GF}(B)$$

$$x^+ = 0.5 \cdot x + w$$

$$x_0 \sim \mathcal{U}(1, 100), \quad w \sim \mathcal{U}(-0.1, 0.1)$$

$$\neg(x < -1) \text{UG}(-1 \leq x \leq 1)$$



$$\text{GF}(\{q_0, q_2\}) \implies \text{GF}(\emptyset)$$

# Deterministic Streett Automata (DSA)

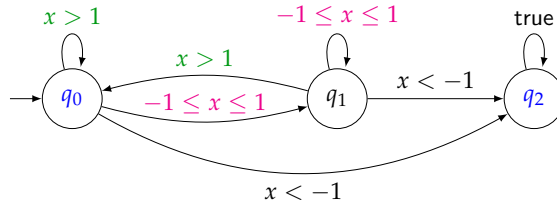
- states  $Q$ , alphabet  $\Sigma$ , transition function  $\delta: Q \times \Sigma \rightarrow Q$
- accepting pair  $(A, B)$  of sets of states,  $A, B \subseteq Q$ , and acceptance condition:

$$\text{GF}(A) \implies \text{GF}(B)$$

$$x^+ = 0.5 \cdot x + w$$

$$x_0 \sim \mathcal{U}(1, 100), \quad w \sim \mathcal{U}(-0.1, 0.1)$$

$$\neg(x < -1) \text{UG}(-1 \leq x \leq 1)$$



$$\text{GF}(\{q_0, q_2\}) \implies \text{GF}(\emptyset)$$

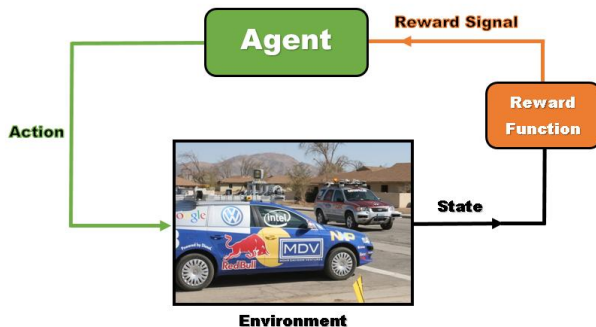
*with rewards? thanks, but no thanks*

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

# Digression on “reward is enough” hypothesis in RL

- RL: **learning** algorithm, relies on **reward signal** from environment
- synthesises **policies** (action selection) maximising **cumulative reward**



- are rewards enough? a matter of feedback ...

# LCRL - Logically Constrained RL

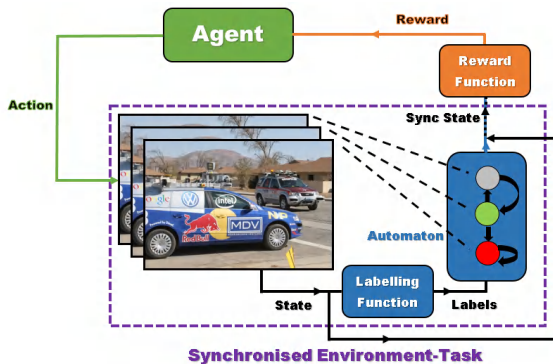
**IN** requirement, task, or objective

- encode **task/objective** (say, from LTL formula) as **automaton**
- **synchronise automaton** with environment
- **synthesise policies** via RL

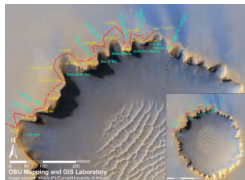
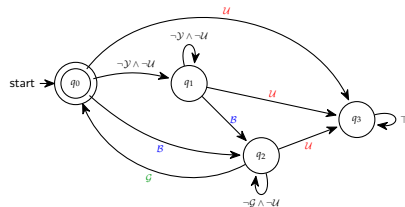
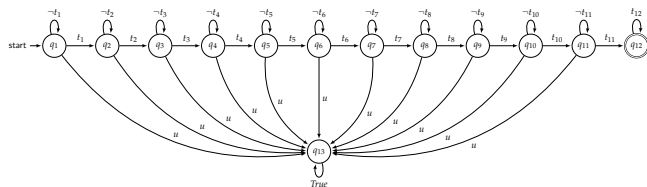
% via labels

% automaton guides exploration

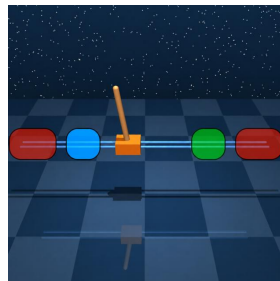
**OUT** certified **policies**: max probability of **task** satisfaction



# LCRL - Logically Constrained RL



“obligation”  
(e.g., sequential-reach, while avoid)

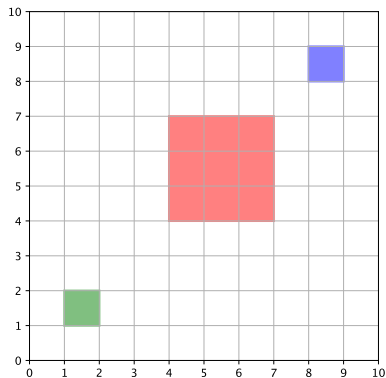


“reactivity”

# Automata-based policy synthesis in RL

environment

LTL specification



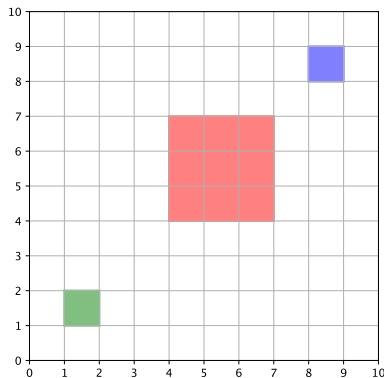
slippery grid world

labels:  $\mathcal{B}$  (blue),  $\mathcal{G}$  (green),  $\mathcal{U}$  (red)

$$GF \mathcal{B} \wedge GF \mathcal{G} \wedge G \neg \mathcal{U}$$

# Automata-based policy synthesis in RL

## environment



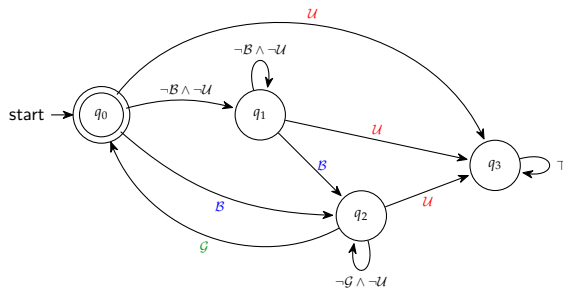
slippery grid world

labels:  $\mathcal{B}$  (blue),  $\mathcal{G}$  (green),  $\mathcal{U}$  (red)

## LTL specification

$$GF \mathcal{B} \wedge GF \mathcal{G} \wedge G \neg \mathcal{U}$$

## automaton (Büchi)



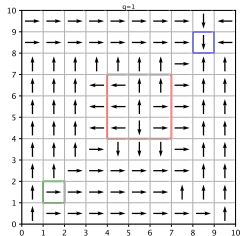
label-dep meaning:

$q_1 \rightarrow$  expecting  $\mathcal{B}$

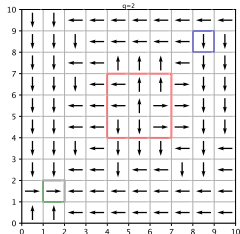
$q_2 \rightarrow$  expecting  $\mathcal{G}$

$q_3 \rightarrow$  stuck at  $\mathcal{U}$

# Learned policy (action map)



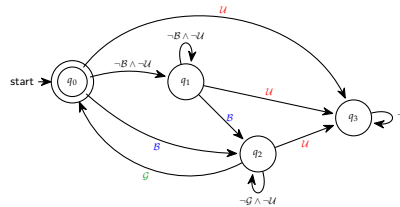
$q_1 \rightarrow$  expecting  $\mathcal{B}$



$q_2 \rightarrow$  expecting  $\mathcal{G}$

## key features:

- $q_1$ : arrows to blue region  $\mathcal{B}$
- $q_2$ : arrows to green region  $\mathcal{G}$
- for both  $q_1, q_2$  (and  $q_0$ ),  
avoid red region  $\mathcal{U}$



- lexico-graphic RL: requirements + performance
- DeepSynth: learning both specification & policy
  
- Cautious RL: requirements at both training- & deployment-time
- BARL-LTL: Bayes-adaptive RL under temporal tasks
- (Bayesian verification)
  
- beyond single tasks: zero-shot task generalisation

*[cf. bib at end of deck]*

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

# Decision problems: SAT and SMT



- SAT is a decision problem (yes/no question)
- problem: to find satisfying assignment of Boolean function
- e.g., assume Boolean  $x_i$ , check 1st-order logic formula

$$\exists x_1, x_2, x_3 : (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge \neg x_1$$

# Decision problems: SAT and SMT



- SAT is a decision problem (yes/no question)
- problem: to find satisfying assignment of Boolean function
- e.g., assume Boolean  $x_i$ , check 1st-order logic formula

$$\exists x_1, x_2, x_3 : (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge \neg x_1$$

- SMT is a decision problem for a logical formula within a theory
- instance: theory of non-linear arithmetics over real closed fields
- e.g., assume reals  $x_i \in \mathbb{R}$ , check

$$\exists x_1, x_2 : x_1 \geq 0 \Rightarrow 3x_1 + 2x_2 + 1 > 0$$

# Decision problems: SAT and SMT



- SAT is a decision problem (yes/no question)
- problem: to find satisfying assignment of Boolean function
- e.g., assume Boolean  $x_i$ , check 1st-order logic formula

$$\exists x_1, x_2, x_3 : (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge \neg x_1$$

- SMT is a decision problem for a logical formula within a theory
- consider **harder problem** (2nd-order logic): assume reals  $x_i \in \mathbb{R}$ ,  
**seek function**  $F : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ , s.t.

$$\exists F(x_1, x_2), \forall x_1, x_2 :$$

$$F(x_1, x_2) \geq x_1 \wedge F(x_1, x_2) \geq x_2 \wedge (F(x_1, x_2) = x_1 \vee F(x_1, x_2) = x_2)$$

(spoiler:  $F(x_1, x_2) = \max\{x_1, x_2\}$ )

neural proofs = 1) proof rules + 2) certificates synthesis

- 1) proof rules
- 2) certificate synthesis

neural proofs = 1) proof rules + 2) certificates synthesis

- 1) proof rules = spec-dependent algebraic conditions on model's dynamics
- 2) certificate synthesis

neural proofs = 1) proof rules + 2) certificates synthesis

- 1) proof rules = spec-dependent algebraic conditions on model's dynamics
- 2) certificate synthesis
  - 2a) deductive
  - 2b) inductive

neural proofs = 1) proof rules + 2) certificates synthesis

- 1) proof rules = spec-dependent algebraic conditions on model's dynamics
- 2) certificate synthesis
  - 2a) deductive
  - 2b) inductive
  - 2c) data-driven

# Lyapunov functions for stability

- $x^+ = f(x)$ , assume  $x_e \in \mathbb{R}^n$  is an equilibrium (the label),  $f(x_e) = x_e$
- ensure (asymptotic) stability of  $x_e \in \mathcal{D} \subseteq \mathbb{R}^n$  ( $\mathcal{D}$  given)
- by finding  $\mathbb{R}$ -valued Lyapunov function  $V(x)$ , satisfying

① lower bound:

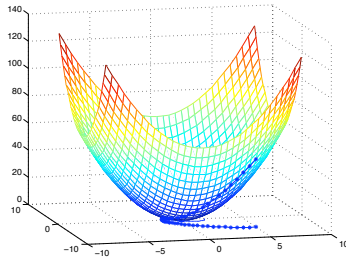
$$V(x_e) = 0 \quad (1)$$

② positive definiteness:

$$V(x) > 0, \quad \forall x \in \mathcal{D} \setminus \{x_e\} \quad (2)$$

③ negative decrease:

$$V(f(x)) - V(x) < -\epsilon, \quad \forall x \in \mathcal{D} \setminus \{x_e\} \quad (\epsilon > 0) \quad (3)$$



# Lyapunov functions for stability

- $x^+ = f(x)$ , assume  $x_e \in \mathbb{R}^n$  is an equilibrium (the label),  $f(x_e) = x_e$
- ensure (asymptotic) stability of  $x_e \in \mathcal{D} \subseteq \mathbb{R}^n$  ( $\mathcal{D}$  given)
- by finding  $\mathbb{R}$ -valued Lyapunov function  $V(x)$ , satisfying

- ❶ lower bound:

$$V(x_e) = 0 \tag{1}$$

- ❷ positive definiteness:

$$V(x) > 0, \forall x \in \mathcal{D} \setminus \{x_e\} \tag{2}$$

- ❸ negative decrease:

$$V(f(x)) - V(x) < -\epsilon, \forall x \in \mathcal{D} \setminus \{x_e\} \quad (\epsilon > 0) \tag{3}$$

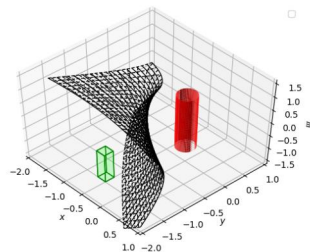
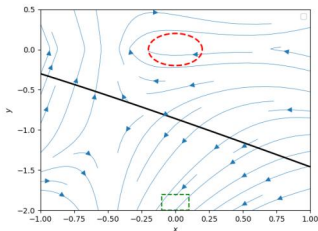
- that is, solve following synthesis problem:

$$\exists V: \mathcal{D} \rightarrow \mathbb{R} \quad \text{s.t.} \quad \forall x \in \mathcal{D}, \quad \text{conditions (1)} \wedge \text{(2)} \wedge \text{(3) hold}$$

- feed 2nd-order logical formula above to SMT solver

# Barrier certificates for safety

- consider  $x^+ = f(x)$ , with sets  $\mathcal{I}$  (initial, label green) and  $\mathcal{U}$  (unsafe, red)
- ensure there exists no trajectory starting in  $\mathcal{I}$  ever entering  $\mathcal{U}$



# Barrier certificates for safety



- consider  $x^+ = f(x)$ , with sets  $\mathcal{I}$  (initial, label green) and  $\mathcal{U}$  (unsafe, red)
- ensure there exists no trajectory starting in  $\mathcal{I}$  ever entering  $\mathcal{U}$
- by finding barrier function  $B(x)$ , satisfying

- consider  $x^+ = f(x)$ , with sets  $\mathcal{I}$  (initial, label green) and  $\mathcal{U}$  (unsafe, red)
- ensure there exists no trajectory starting in  $\mathcal{I}$  ever entering  $\mathcal{U}$
- by finding barrier function  $B(x)$ , satisfying

- 1 negativity within initial set  $\mathcal{I}$ :

$$B(x) \leq 0, \forall x \in \mathcal{I}$$

- 2 positivity within unsafe set  $\mathcal{U}$ :

$$B(x) > 0, \forall x \in \mathcal{U}$$

- 3 set invariance property via negative decrease:

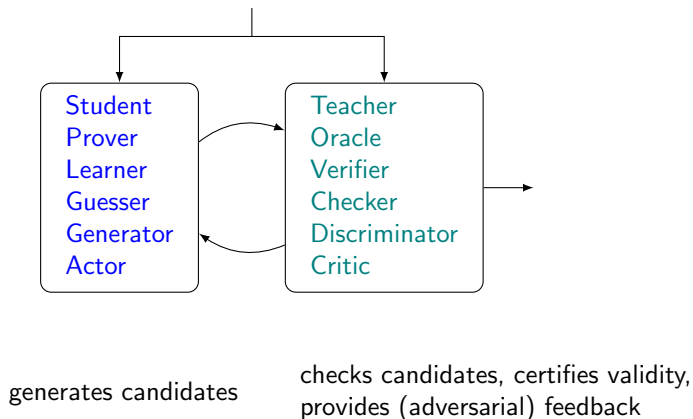
$$B(f(x)) - B(x) < 0, \forall x \in (\mathcal{U} \cup \mathcal{I})^c$$

- again, feed a 2nd-order logical formula to SMT solver

- consider probabilistic program  $x^+ = f(x, w)$ :

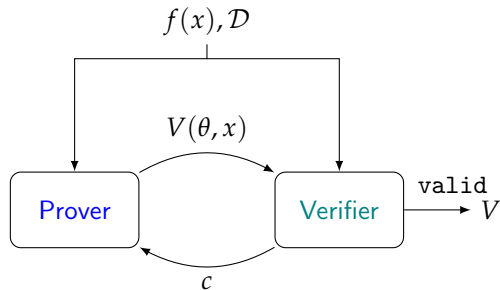
```
while (x > 0):  
  if prob(0.75) {  
    x-- }
```

- ensure that, for any  $x \geq 0$ ,  $F(x \leq 0)$  holds *almost surely (with probability 1)*
- by finding ranking function  $V(x)$ , satisfying
  - 1  $V(0) = 0$
  - 2  $x > 0 \rightarrow V(x) > 0$
  - 3  $x > 0 \rightarrow \mathbb{E}[V(f(x))] \leq V(x) - \epsilon$   
namely, if  $x > 0$ ,  $V(x)$  *decreases in expectation* by  $\epsilon > 0$
- again, feed a 2nd-order logical formula to SMT solver



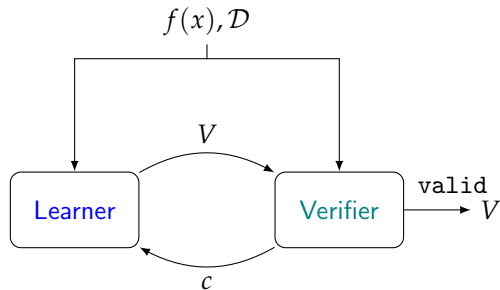
- can be framed as *oracle-based synthesis*, as *infinitely-constrained optimisation*

# Counterexample-guided Inductive Synthesis (CEGIS)



- inductive synthesis loop

1. sample (finite) set  $S \subset \mathcal{D}$
2. **Prover** generates  $V(\theta, \cdot)$  via query SMT solver on formula:  
 $\exists \theta \in \Theta : \text{proof rules hold on points } s \in S$
3. **Verifier** checks either  $V(\cdot, x)$  valid over dense  $\mathcal{D}$ , or counterexample  $c$  :  
query SMT solver on formula  $\exists c \in \mathcal{D} : \text{a proof rule does not hold on } c$
4.  $S \leftarrow S \cup c$ , loop back to 2



- sound, but might not end: infinite
  - search space  $(\theta \in \Theta)$  &
  - domain  $(x \in \mathcal{D})$

- inductive synthesis loop

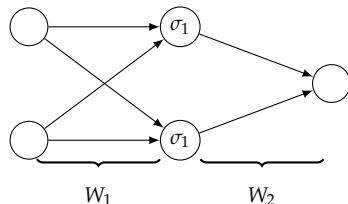
1. sample (finite) set  $S \subset \mathcal{D}$
2. Learner trains neural  $V(\theta, \cdot)$  via loss function  $L$ ;  
 $L$  penalises deviations from conditions in proof rules
3. Verifier checks either  $V(\cdot, x)$  valid over dense  $\mathcal{D}$ , or counterexample  $c$  :  
query SMT solver on formula  $\exists c \in \mathcal{D} : \text{a proof rule does not hold on } c$
4.  $S \leftarrow S \cup c$ , loop back to 2

# Neural networks as Lyapunov functions

- neural nets are general and flexible  
(universal function approximators)
- Learner trains shallow neural network

$$V(\theta, x) = W_2 \cdot \sigma_1(W_1 x + b_1)$$

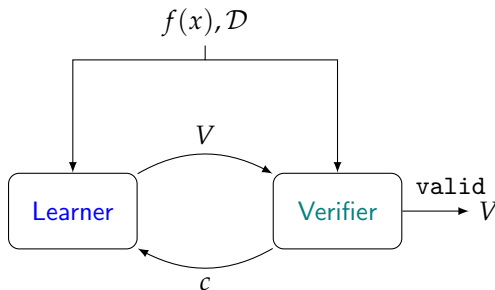
( $\theta$  has weights  $W_i$ , biases  $b_i$ ,  $\sigma_1$  activations)



- loss function enforces Lyapunov conditions in (1)+(2) and (3) on points in  $S$ :

$$L(S) = |V(0)| + \sum_{s \in S} \max\{0, -V(s)\} + \sum_{s \in S} \max\{0, V(f(s)) - V(s) + \epsilon\}$$

- loss function  $L$  is “pretty good” proxy of proof rules, penalises deviations

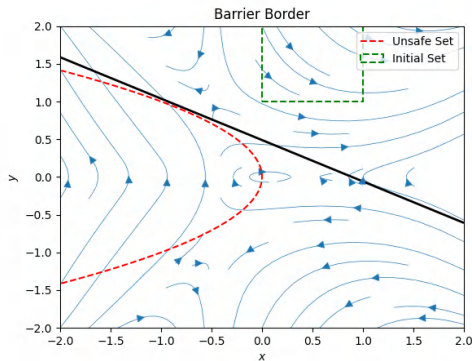
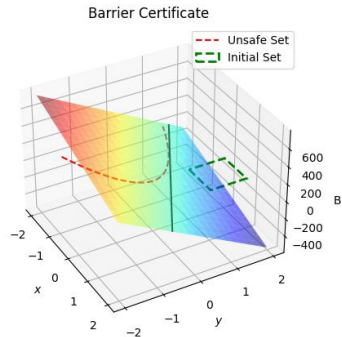


- loss function enforces Lyapunov conditions in (1)+(2) and (3) on points in  $S$ :

$$L(S) = |V(0)| + \sum_{s \in S} \max\{0, -V(s)\} + \sum_{s \in S} \max\{0, V(f(s)) - V(s) + \epsilon\}$$

- loss function  $L$  is “pretty good” proxy of proof rules, penalises deviations
- **surprisingly effective!** Information exchange **Learner**  $\leftrightarrow$  **Verifier** is crucial

# Synthesis of barrier certificates - examples

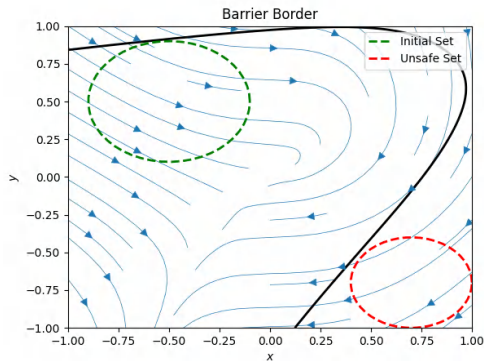
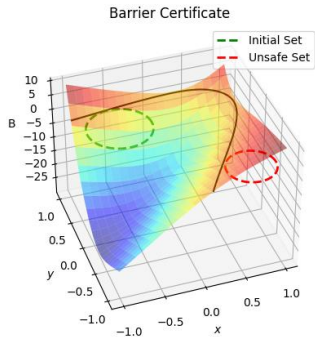


$$\begin{cases} x^+ &= y + 2xy \\ y^+ &= -x + 2x^2 - y^2 \end{cases}$$

[10] · Linear

- comparable to SOS

# Synthesis of barrier certificates - examples

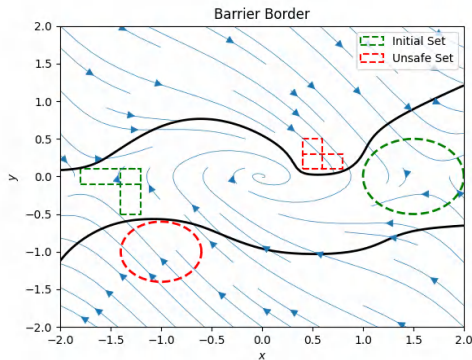
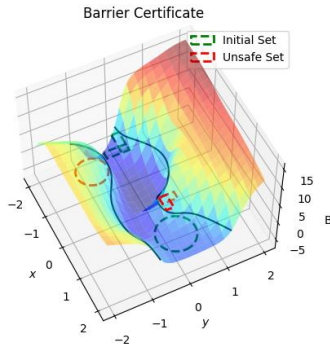


$$\begin{cases} x^+ &= \exp(-x) + y - 1 \\ y^+ &= -\sin(x)^2 \end{cases}$$

[20] · Softplus

- SOS not applicable

# Synthesis of barrier certificates - examples



$$\begin{cases} x^+ &= y \\ y^+ &= -x - y + \frac{1}{3}x^3 \end{cases}$$

$[20, 20] \cdot \text{Sigmoid}, \text{Sigmoid}$

- SOS must over-convexify constraints (conservative? scalable?)

- dynamical models with **inputs** (i.e., external non-determinism)

$$x^+ = f(x, \mathbf{u})$$

→ synthesis of “**control** certificates”

- modify known synthesis problem:

$$\exists V: \mathcal{D} \rightarrow \mathbb{R} \quad \text{s.t.} \quad \forall x \in \mathcal{D} \quad \text{conditions (1)} \wedge \text{(2)} \wedge \text{(3) hold}$$

- dynamical models with inputs (i.e., external non-determinism)

$$x^+ = f(x, u)$$

→ synthesis of “control certificates”

- approach:

- 1 control policies are also NN-templated
- 2 concurrent synthesis of controls + corresponding certificates

# Synthesis of **control** certificates

- dynamical models with **inputs** (i.e., external non-determinism)

$$x^+ = f(x, \mathbf{u})$$

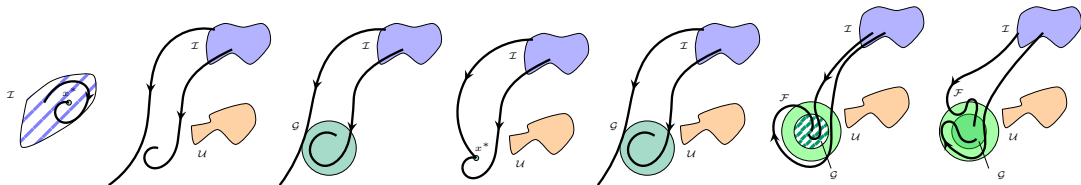
→ synthesis of “**control** certificates”

- (back to) broad class of safe/co-safe properties

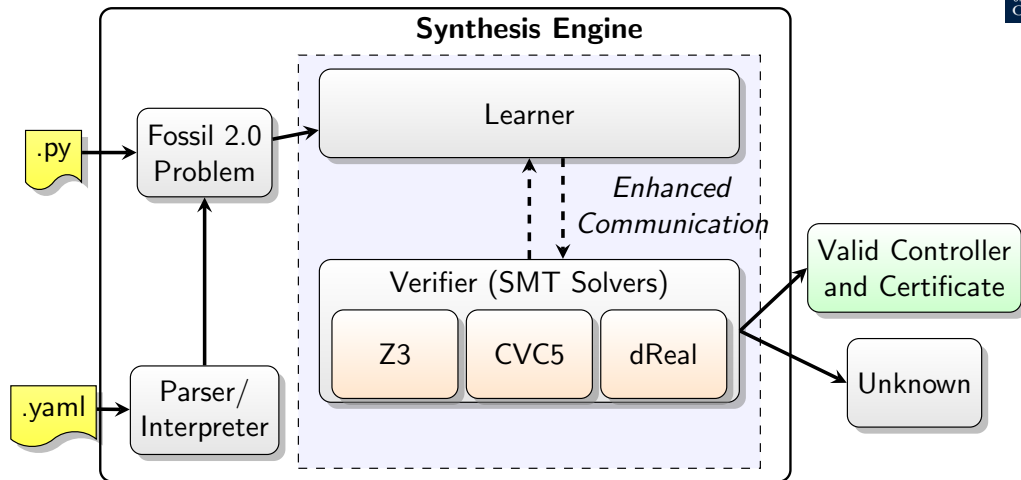
$$\forall x_0 \in \mathcal{I}, \quad \exists T \in \mathbb{N}, \\ x_T \in \mathcal{G},$$

$$\forall t \in \{0, \dots, T-1\}, \\ x_t \notin \mathcal{U},$$

$$\forall \tau \geq T : \\ x_\tau \in \mathcal{F}$$



|    | $N_s$ | $N_u$ | Property  | Neurons        | Activations                                                          | $T$ (s)                 |               |                | Success (%) |
|----|-------|-------|-----------|----------------|----------------------------------------------------------------------|-------------------------|---------------|----------------|-------------|
|    |       |       |           |                |                                                                      | min                     | $\mu$         | max            | S           |
| 1  | 2     | 0     | Stability | [6]            | $[\varphi_2]$                                                        | 0.01 ( $\approx 0.00$ ) | 0.16 (0.15)   | 1.50 (1.48)    | 100         |
| 2  | 3     | 0     | Stability | [8]            | $[\varphi_2]$                                                        | 0.28 ( $\approx 0.00$ ) | 2.22 (0.45)   | 12.57 (3.31)   | 100         |
| 3  | 2     | 2     | Stability | [4]            | $[\varphi_2]$                                                        | 0.07 (0.01)             | 0.19 (0.02)   | 0.47 (0.04)    | 100         |
| 4  | 2     | 2     | Stability | [5]            | $[\varphi_2]$                                                        | 0.09 (0.01)             | 0.26 (0.02)   | 0.54 (0.03)    | 100         |
| 5  | 2     | 0     | ROA       | [5]            | $[\sigma_{\text{soft}}]$                                             | 0.21 (0.12)             | 14.09 (12.59) | 25.32 (22.13)  | 40          |
| 6  | 3     | 3     | ROA       | [8]            | $[\varphi_2]$                                                        | 1.24 (0.02)             | 39.08 (0.03)  | 287.89 (0.04)  | 100         |
| 7  | 2     | 0     | Safety    | [15]           | $[\sigma_t]$                                                         | 0.44 (0.35)             | 3.36 (2.90)   | 7.61 (7.11)    | 100         |
| 9  | 8     | 0     | Safety    | [10]           | $[\varphi_1]$                                                        | 12.63 (7.71)            | 51.97 (32.75) | 70.59 (44.66)  | 70          |
| 10 | 3     | 1     | Safety    | [15]           | $[\sigma_t]$                                                         | 1.57 (0.19)             | 11.87 (2.50)  | 51.08 (7.52)   | 90          |
| 11 | 3     | 0     | SWA       | [6], [5]       | $[\varphi_2], [\sigma_t]$                                            | 0.19 (0.05)             | 2.46 (0.100)  | 12.10 (0.20)   | 90          |
| 12 | 2     | 0     | SWA       | [5], [5, 5]    | $[\varphi_2], [\sigma_{\text{sig}}, \varphi_2]$                      | 0.13 (0.06)             | 0.27 (0.14)   | 0.39 (0.20)    | 100         |
| 13 | 2     | 1     | SWA       | [8], [5]       | $[\varphi_2], [\varphi_2]$                                           | 0.06 (0.03)             | 0.20 (0.10)   | 0.58 (0.24)    | 90          |
| 14 | 3     | 1     | SWA       | [10], [8]      | $[\varphi_2], [\sigma_t]$                                            | 4.06 (0.87)             | 19.81 (2.73)  | 103.49 (7.23)  | 90          |
| 15 | 2     | 0     | RWA       | [4]            | $[\varphi_2]$                                                        | 0.14 (0.09)             | 1.81 (1.75)   | 4.70 (4.63)    | 100         |
| 16 | 3     | 0     | RWA       | [16]           | $[\varphi_2]$                                                        | 1.36 (0.09)             | 14.10 (0.14)  | 72.97 (0.20)   | 90          |
| 17 | 2     | 1     | RWA       | [4, 4]         | $[\sigma_{\text{sig}}, \varphi_2]$                                   | 0.59 (0.27)             | 6.82 (3.32)   | 20.07 (11.46)  | 100         |
| 18 | 3     | 1     | RWA       | [5]            | $[\varphi_2]$                                                        | 0.46 (0.11)             | 16.06 (5.81)  | 72.47 (44.64)  | 80          |
| 19 | 2     | 2     | RWA       | [5]            | $[\sigma_{\text{sig}}]$                                              | 0.69 (0.40)             | 1.38 (0.94)   | 2.14 (1.90)    | 100         |
| 20 | 2     | 0     | RSWA      | [4]            | $[\varphi_2]$                                                        | 0.19 (0.03)             | 1.29 (1.04)   | 3.79 (3.37)    | 100         |
| 21 | 3     | 0     | RSWA      | [16]           | $[\varphi_2]$                                                        | 4.81 (0.13)             | 27.14 (0.19)  | 80.95 (0.25)   | 100         |
| 22 | 2     | 0     | RSWA      | [5, 5]         | $[\sigma_{\text{sig}}, \varphi_2]$                                   | 1.52 (0.06)             | 4.45 (0.19)   | 10.97 (0.35)   | 100         |
| 23 | 2     | 1     | RSWA      | [8]            | $[\varphi_2]$                                                        | 0.21 (0.05)             | 0.67 (0.25)   | 1.19 (0.91)    | 100         |
| 24 | 2     | 2     | RSWA      | [5, 5]         | $[\sigma_{\text{sig}}, \varphi_2]$                                   | 0.98 (0.16)             | 1.23 (0.28)   | 1.61 (0.46)    | 100         |
| 25 | 2     | 0     | RAR       | [6], [6]       | $[\sigma_{\text{soft}}], [\varphi_2]$                                | 6.65 (1.08)             | 24.74 (6.46)  | 77.80 (15.06)  | 100         |
| 26 | 2     | 2     | RAR       | [6, 6], [6, 6] | $[\sigma_{\text{sig}}, \varphi_2], [\sigma_{\text{sig}}, \varphi_2]$ | 5.13 (1.34)             | 26.99 (9.90)  | 101.23 (60.14) | 100         |



[github.com/oxford-oxcav/fossil](https://github.com/oxford-oxcav/fossil)



# A messy business: certificates as supermartingales



- almost-sure (w.p. 1) countable-state program termination (cf. above)
- can be extended to
  - continuous/dense state spaces  
probabilistic programs and general MDPs
  - control/policy/strategy synthesis
  - quantitative termination
    - via supermartingale inequalities
    - via 'decomposition' result (cf bib)

- almost-sure (w.p. 1) countable-state program termination (cf. above)
- can be extended to
  - continuous/dense state spaces  
probabilistic programs and general MDPs
  - control/policy/strategy synthesis
  - quantitative termination
    - via supermartingale inequalities
    - via 'decomposition' result (cf bib)

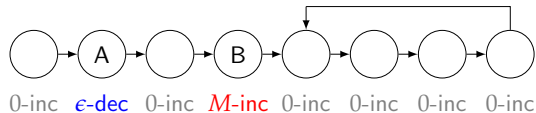
... towards whole-LTL quantitative model checking and design of gMDP

# Proof rule for almost-sure Streett acceptance

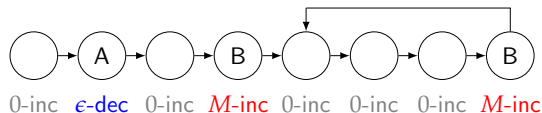
- consider Markov chain  $x^+ = f(x, w)$ , along with **Streett pair**:

$$GF(A) \implies GF(B)$$

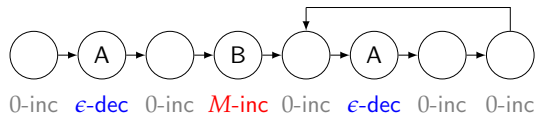
$$A, B \subseteq \mathcal{D}$$



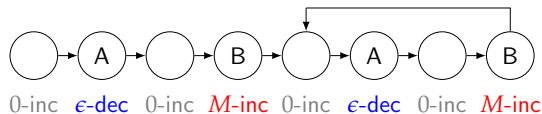
finite A's  
finite B's



finite A's  
infinite B's



infinite A's  
finite B's



infinite A's  
infinite B's

# Proof rule for almost-sure Streett acceptance

- consider Markov chain  $\mathbf{x}^+ = f(\mathbf{x}, \mathbf{w})$ , along with **Streett pair**:

$$\text{GF}(A) \implies \text{GF}(B)$$

$$A, B \subseteq \mathcal{D}$$

## Streett Supermartingale Proof Rule [AA et al, CAV24]

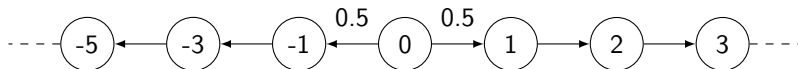
$\exists$  function  $V : \mathcal{D} \rightarrow \mathbb{R}_{\geq 0}$ ,

- In  $B$ :** allow  $V$  to **increase** in expectation **by  $M$**  (positive const)
- In  $A \setminus B$ :** **decrease**  $V$  in expectation **by  $\epsilon$**  (positive const)
- Otherwise:** ensure  $V$  does not increase in expectation

$V$  is **non-negative** and **almost** a supermartingale

allow increase in  $B$





$$\Phi = \text{FG}(x < 0) \vee \text{GF}(x \text{ is even})$$

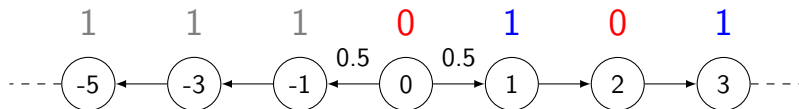
$\text{FG}(x < 0)$   
with probability 0.5

$\text{GF}(x \text{ is even})$   
with probability 0.5

but  $\Phi$  holds with probability 1 !

- in literature, ‘insufficient’ certificates, only useful for  
almost-sure recurrence  
a.s. persistence

# Streett supermartingale - example



$$\Phi = \text{GF}(\underbrace{x \geq 0}_A) \implies \text{GF}(\underbrace{x \text{ is even}}_B)$$

$B$ :  $x$  is even: allow  $V$  to **increase** in expectation by  $M = 1$

$A \setminus B$ :  $x \geq 0$  and  $x$  is odd: **decrease**  $V$  in expectation by  $\epsilon = 1$

$(A \cup B)^c$ :  $x < 0$  and  $x$  is odd: ensure  $V$  does not increase in expectation

$\implies \Phi$  holds with probability 1

- can be extended to dense-space models, to control synthesis, ...

- I - invariance generation
- V - verification (certificate synthesis)
- C - control synthesis

| Benchmark        | $\omega$ -Regular Specification                                      | Output | Time [s] |
|------------------|----------------------------------------------------------------------|--------|----------|
| EvenOrNegative   | $FG(x < 0) \vee GF(x \text{ even})$                                  | V      | 0.09     |
| SafeRWalk1       | $G(x < 100)$                                                         | VIC    | 1.09     |
| PersistRW        | $FG(x \leq 10)$                                                      | VI     | 1.16     |
| RecurRW          | $GF(x > 100)$                                                        | VI     | 1.49     |
| SafeRWalk2       | $G(x \geq 10)$                                                       | VIC    | 1.09     |
| GuaranteeRW      | $G(x \geq -10) \rightarrow F(x \geq 10^3)$                           | VI     | 5.61     |
| Temperature1     | $FG(\neg \text{Hot} \wedge \neg \text{Cold})$                        | VIC    | 4.11     |
| Temperature2     | $GF(x \leq 30) \wedge G(x \leq 60)$                                  | VI     | 28.93    |
| Temperature3     | $G(\text{Safe}) \wedge [GF(\text{Cold}) \rightarrow GF(\text{Hot})]$ | VIC    | 28.58    |
| Temperature4     | $G(\text{Safe}) \wedge [GF(\text{Cold}) \rightarrow GF(\text{Hot})]$ | VC     | 4.64     |
| FinMemoryControl | $GF(x \leq 0) \rightarrow GF(x \geq 100)$                            | VIC    | 16.73    |

so far we have seen that:

- ranking supermartingales for almost-sure reachability
- almost sure persistence, or a.s. recurrence
- Streett supermartingales for almost-sure reactivity conditions  
and indeed **many other similar proof rules** ...

so far we have seen that:

- ranking supermartingales for almost-sure reachability
- almost sure persistence, or a.s. recurrence
- Streett supermartingales for almost-sure reactivity conditions  
and indeed **many other similar proof rules** ...
- restrict state space to **supporting invariant**  $I \subseteq \text{State}$ , so that:

$$\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$$

so far we have seen that:

- ranking supermartingales for almost-sure reachability
- almost sure persistence, or a.s. recurrence
- Streett supermartingales for almost-sure reactivity conditions  
and indeed **many other similar proof rules** ...
- restrict state space to **supporting invariant**  $I \subseteq \text{State}$ , so that:

$$\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$$

- now, we can generalise any **almost-sure proof rule** to **quantitative** supermartingale certificates:

$$\mathbb{P}(\text{Specification}) \geq p$$

- this is done via a “decomposition” result (next)

- soundness of decomposition [AA et al, CAV25] - suppose:
  - ①  $\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$
  - ②  $\mathbb{P}(\mathbf{G} I) \geq p$
- then  $\mathbb{P}(\text{Specification}) \geq p$

- soundness of decomposition [AA et al, CAV25] - suppose:
  - ①  $\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$
  - ②  $\mathbb{P}(\mathbf{G} I) \geq p$
- then  $\mathbb{P}(\text{Specification}) \geq p$
  
- special case:
  - ① if  $\mathbb{P}(\text{Streett acceptance} \mid \mathbf{G} I) = 1$   
(e.g., via Streett supermartingale & supporting invariant, [AA et al, CAV'24])
  - ② if  $\mathbb{P}(\mathbf{G} I) \geq p$   
(e.g., via stochastic invariant (supermartingale) [Kushner, 1965] or repulsing supermartingales [Chatterjee et.al., POPL'17][Takisaka et.a., ATVA'18])
- then  $\mathbb{P}(\text{Streett acceptance}) \geq p$

- assume  $\text{State} = \mathbb{R}^k$ , introduce *parametric templates* for supermartingale certificates and supporting invariants, consider:

$$\exists \text{ parameter } \forall \text{ states: Streitt supermartingale proof rule}$$

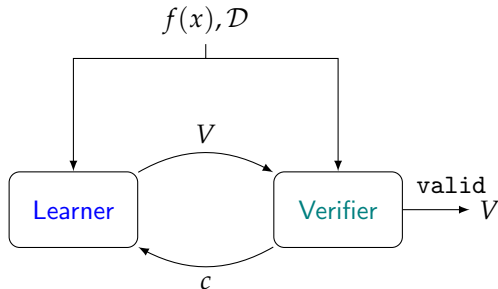
- sound and relatively-complete reduction to SMT solving via applying Farkas' Lemma / Positivstellensatz

| Benchmark                       | $\omega$ -Regular Specification                          | Attained Bounds | Time [s] |
|---------------------------------|----------------------------------------------------------|-----------------|----------|
| Gambler's Ruin ( $d = 2, a^x$ ) | $\text{GF}(x = 0)$                                       | [0.380, 0.671]  | 8.64     |
| Gambler's Ruin ( $d = 3, a^x$ ) | $\text{GF}(x = 0)$                                       | [0.545, 0.671]  | 9.87     |
| Gambler's Ruin ( $d = 4, a^x$ ) | $\text{GF}(x = 0)$                                       | [0.601, 0.671]  | 24.02    |
| Gambler's Ruin ( $d = 5, a^x$ ) | $\text{GF}(x = 0)$                                       | [0.621, 0.671]  | 88.17    |
| BecomingRichOnce ( $d = 2$ )    | $(x \geq 10) \cup (x \geq 100)$                          | [0.610, 1.000]  | 4.68     |
| BecomingRichOnce ( $d = 3$ )    | $(x \geq 10) \cup (x \geq 100)$                          | [0.709, 0.974]  | 7.39     |
| BecomingRichOnce ( $d = 4$ )    | $(x \geq 10) \cup (x \geq 100)$                          | [0.776, 0.908]  | 12.15    |
| BecomingRichOnce ( $d = 5$ )    | $(x \geq 10) \cup (x \geq 100)$                          | [0.807, 0.880]  | 59.04    |
| Reactivity1 ( $d = 2$ )         | $\text{GF}(x \leq 6) \rightarrow \text{GF}(x \leq 0)$    | [0.166, 0.166]  | 8.18     |
| Reactivity2 ( $d = 2$ )         | $\text{GF}(x \leq 10) \rightarrow \text{GF}(x \geq 100)$ | [0.250, 0.250]  | 8.26     |

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

# Constructing certificates - scaling up CEGIS

- CEGIS is bottle-necked at the **Verifier stage** (calls to SMT solver)



- CEGIS is bottle-necked at the **Verifier stage** (calls to SMT solver)
- consider NL vector field  $f(x), x \in \mathcal{X}$
- partition domain  $\mathcal{X} = \cup_i X_i, \quad X_i \cap X_j = \emptyset$
- **1<sup>st</sup>-order** Taylor expand  $f$  locally:  $\forall x \in X_i, \tilde{f}(x) = A_i x + d, \|d\| \leq \delta_i$

- CEGIS is bottle-necked at the **Verifier stage** (calls to SMT solver)
- consider NL vector field  $f(x), x \in \mathcal{X}$
- partition domain  $\mathcal{X} = \cup_i X_i, \quad X_i \cap X_j = \emptyset$
- **1<sup>st</sup>-order** Taylor expand  $f$  locally:  $\forall x \in X_i, \tilde{f}(x) = A_i x + d, \|d\| \leq \delta_i$
  
- consider **PWA template** for certificate, e.g. as **ReLU-NN**:  $\forall x \in X_i, V(x) = L_i x$
- consider local  $\delta_i$ -robust proof rules over  $\tilde{f}$  in  $X_i$ , say rule-3 for Lyapunov:
$$V(\tilde{f}(x)) - V(x) = L_i A_i x - L_i x < -(\epsilon + L_i \delta_i), \quad \forall x \in X_i$$
- leverage faster **linear arithmetics** solvers (eg,  $\alpha, \beta$ -CROWN, Marabou)

- CEGIS is bottle-necked at the **Verifier stage** (calls to SMT solver)
- consider NL vector field  $f(x), x \in \mathcal{X}$
- partition domain  $\mathcal{X} = \cup_i X_i, \quad X_i \cap X_j = \emptyset$
- 1<sup>st</sup>-order Taylor expand  $f$  locally:  $\forall x \in X_i, \tilde{f}(x) = A_i x + d, \|d\| \leq \delta_i$
- consider **PWA template** for certificate, e.g. as **ReLU-NN**:  $\forall x \in X_i, V(x) = L_i x$
- consider local  $\delta_i$ -robust proof rules over  $\tilde{f}$  in  $X_i$ , say rule-3 for Lyapunov:

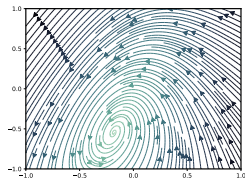
$$V(\tilde{f}(x)) - V(x) = L_i A_i x - L_i x < -(\epsilon + L_i \delta_i), \quad \forall x \in X_i$$

- leverage faster **linear arithmetics** solvers (eg,  $\alpha, \beta$ -CROWN, Marabou)

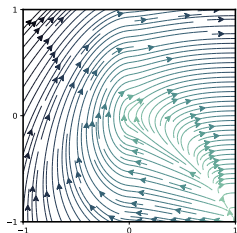
[cf. L4DC26 paper]

# Modern hybridisations: *neural abstractions*

- consider NL vector field  $f(x), x \in \mathcal{X}$
- partition domain  $\mathcal{X} = \cup_i X_i, \quad X_i \cap X_j = \emptyset$
- 1<sup>st</sup>-order Taylor expand  $f$  locally:  $\forall x \in X_i, \tilde{f}(x) = A_i x + d, \|d\| \leq \delta_i$

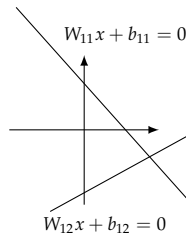
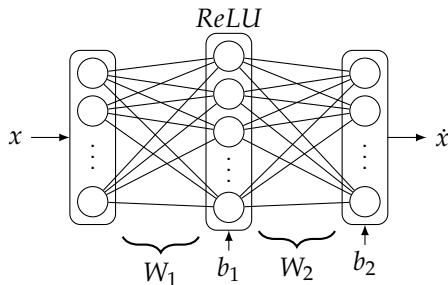


$$\begin{cases} x^+ &= -y - 1.5x^2 - 0.5x^3 - 0.5 \\ y^+ &= 3x - y \end{cases}$$

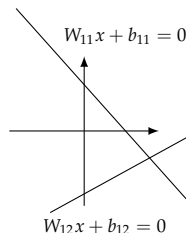
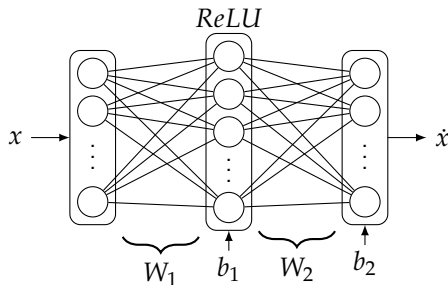


$$\begin{cases} x^+ &= x^2 + y \\ y^+ &= \sqrt[3]{x^2} - x \end{cases}$$

# Modern hybridisations: *neural abstractions*



- $\tilde{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$   $\delta$ -approximates  $f$   
(where  $\delta = \max_i \delta_i$ )
- **ReLU-neural network**  $\tilde{f}$  as  $\delta$ -abstraction of nonlinear vector field  $f$



- synthesis of neural abstraction via CEGIS

- ① learn parameters of ReLU-NN  $\tilde{f}$  w/ MSE loss  $\mathcal{L} = \|f(S) - \tilde{f}(S)\|$ ,  $S$  finite

- ② SMT solver formally checks upper bound  $\delta$  on approximation error:

$$\exists c \in \mathcal{X} \text{ s.t. } \|f(c) - \tilde{f}(c)\| > \delta$$

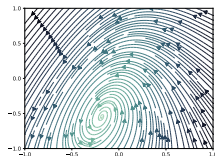
- obtain abstraction as

$$x^+ = \tilde{f}(x) + d, \quad \|d\| \leq \delta, \quad x \in \mathcal{X}$$

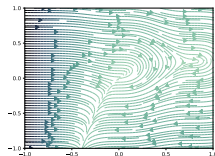
# Neural abstractions - examples

concrete model

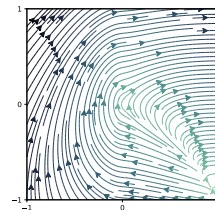
$$\begin{cases} x^+ &= y - 1.5x^2 - 0.5x^3 \\ y^+ &= 3x - y \end{cases}$$



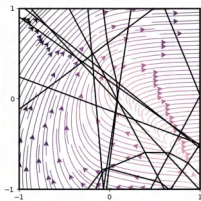
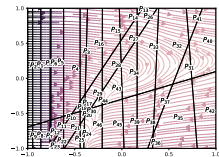
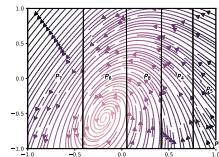
$$\begin{cases} x^+ &= \exp(-x) + y - 1 \\ y^+ &= -\sin(x)^2 \end{cases}$$



$$\begin{cases} x^+ &= x^2 + y \\ y^+ &= \sqrt[3]{x^2} - x \end{cases}$$



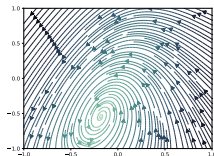
neural abstraction



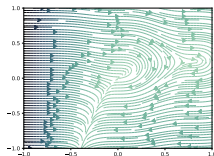
# Neural abstractions - examples

concrete model

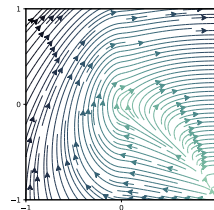
$$\begin{cases} x^+ &= y - 1.5x^2 - 0.5x^3 \\ y^+ &= 3x - y \end{cases}$$



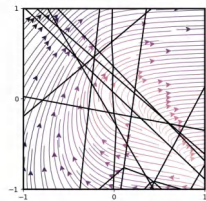
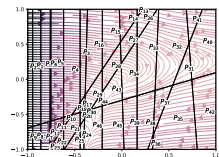
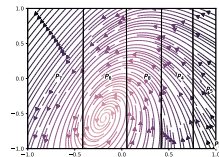
$$\begin{cases} x^+ &= \exp(-x) + y - 1 \\ y^+ &= -\sin(x)^2 \end{cases}$$



$$\begin{cases} x^+ &= x^2 + y \\ y^+ &= \sqrt[3]{x^2} - x \end{cases}$$



neural abstraction



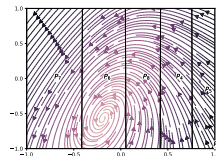
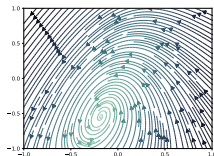
# Neural abstractions - examples

concrete model

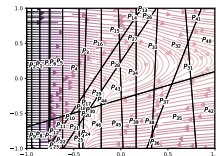
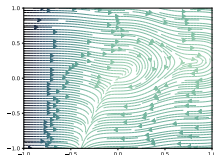
neural abstraction



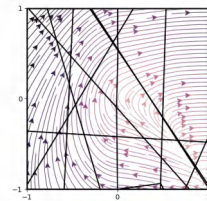
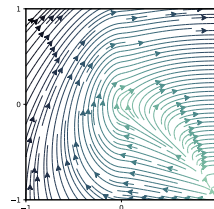
$$\begin{cases} x^+ &= y - 1.5x^2 - 0.5x^3 \\ y^+ &= 3x - y \end{cases}$$



$$\begin{cases} x^+ &= \exp(-x) + y - 1 \\ y^+ &= -\sin(x)^2 \end{cases}$$



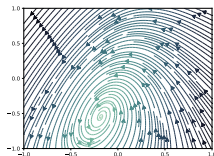
$$\begin{cases} x^+ &= x^2 + y \\ y^+ &= \sqrt[3]{x^2} - x \end{cases}$$



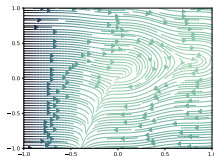
# Neural abstractions - examples

concrete model

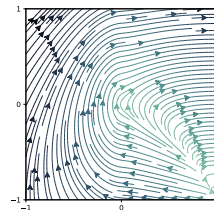
$$\begin{cases} x^+ &= y - 1.5x^2 - 0.5x^3 \\ y^+ &= 3x - y \end{cases}$$



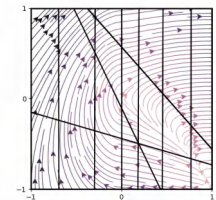
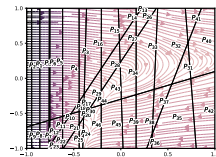
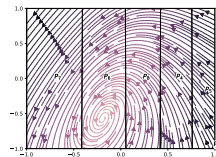
$$\begin{cases} x^+ &= \exp(-x) + y - 1 \\ y^+ &= -\sin(x)^2 \end{cases}$$



$$\begin{cases} x^+ &= x^2 + y \\ y^+ &= \sqrt[3]{x^2} - x \end{cases}$$



neural abstraction



- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

# Alternatives to inductive certificate synthesis



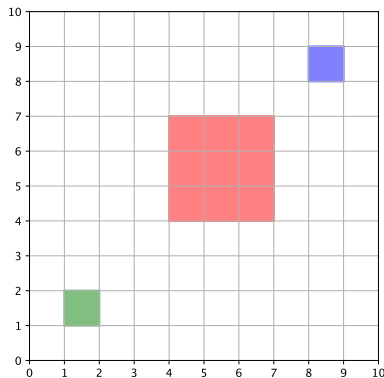
- flexible architecture to synthesise both controllers & certificates
- how about reinforcement learning?

- flexible architecture to synthesise both controllers & certificates
  - how about reinforcement learning?
- 1 consider simple spec (say, [safety](#))
  - 2 set rewards as intuitive
  - 3 train policy via generalist RL algorithm (say, off-policy Q learning)
  - 4 extract value function (VF)

- flexible architecture to synthesise both controllers & certificates
  - how about reinforcement learning?
- 1 consider simple spec (say, [safety](#))
  - 2 set rewards as intuitive
  - 3 train policy via generalist RL algorithm (say, off-policy Q learning)
  - 4 extract value function (VF)
- might (asymptotically optimal) VF be a supermartingale certificate for [safety](#)?

# Automata-based policy synthesis in RL

environment



slippery grid world

labels:  $\mathcal{B}$  (blue),  $\mathcal{G}$  (green),  $\mathcal{U}$  (red)

label-dep meaning:

$q_1 \rightarrow$  expecting  $\mathcal{B}$

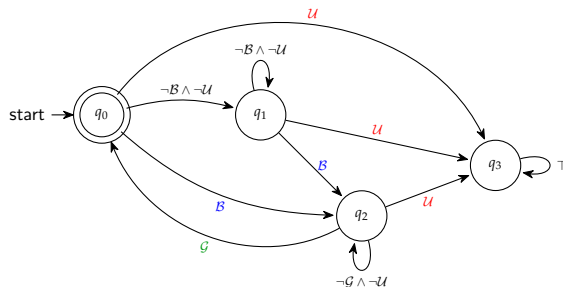
$q_2 \rightarrow$  expecting  $\mathcal{G}$

$q_3 \rightarrow$  stuck at  $\mathcal{U}$

LTL specification - recurrence + safety

$$GF \mathcal{B} \wedge GF \mathcal{G} \wedge G \neg \mathcal{U}$$

automaton (Büchi)

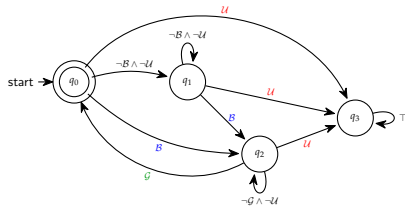


# Intuitive reward shaping from automaton

- reward on automaton states

(fed back over product MDP):

$$r(q) = \begin{cases} +5 & q_0 \text{ (accepting node)} \\ -10 & q_3 \text{ (rejecting sink)} \\ 0 & q_1, q_2 \text{ (leading to } q_0) \end{cases}$$



## Proof rules for a.s. recurrence - $G \neg T$

(r1)  $\exists m > 0, \forall x \in T, W(x) \geq m$

lower bnd. in  $T$

(r2)  $\exists \varepsilon > 0, \forall x \in X \setminus T, \mathbb{E}^\pi W(x') \leq W(x) - \varepsilon$

drift outside  $T$

(r3)  $\exists H, \forall x \in T, \mathbb{E}^\pi W(x') \leq W(x) + H$

drift inside  $T$

## Proof rules for a.s. safety - $G \neg U$

(r1)  $\exists M > 0, \forall x \in U, W(x) \leq M$

upper bnd. in  $U$

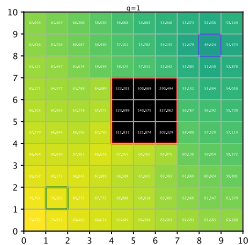
(r2)  $\exists m < M, \forall x \in X \setminus U, W(x) \leq m$

lower bnd. in  $X \setminus U$

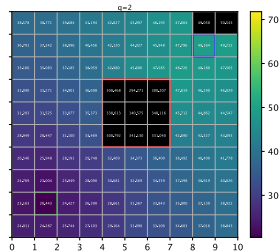
(r3)  $\exists \varepsilon > 0, \forall x \in X \setminus U, \mathbb{E}^\pi W(x') \leq W(x) - \varepsilon$

drift outside  $U$

# Value functions from RL as proof certificates



$q_1$  (expect  $\mathcal{B}$ )



$q = 2$  (expect  $\mathcal{G}$ )

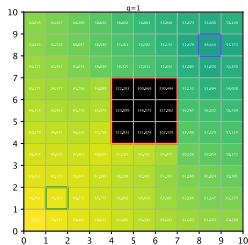
- from Q-function to certificate  $W$

$$\pi(x) = \arg \max_a Q(x, a)$$

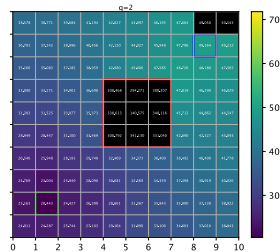
$$W(x) = \max_{x', a'} Q(x', a') - \max_a Q(x, a)$$

- $W$  satisfies proof rules for **recurrence + safety**

# Value functions from RL as proof certificates



$q_1$  (expect  $\mathcal{B}$ )



$q = 2$  (expect  $\mathcal{G}$ )

- from Q-function to certificate  $W$

$$\pi(x) = \arg \max_a Q(x, a)$$

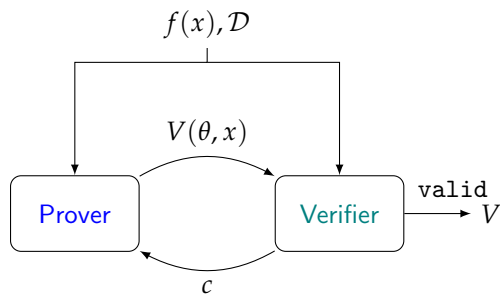
$$W(x) = \max_{x', a'} Q(x', a') - \max_a Q(x, a)$$

- $W$  satisfies proof rules for **recurrence + safety**
- extension to general tasks as **Streett automata**,

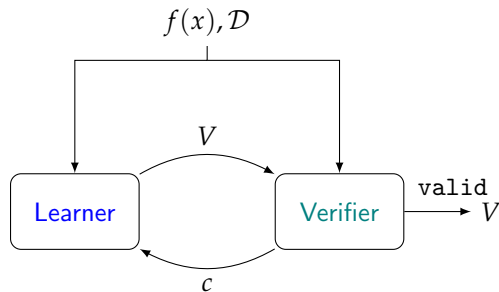
$$\text{GF}(A) \implies \text{GF}(B)$$

[cf. SAIV26 paper]

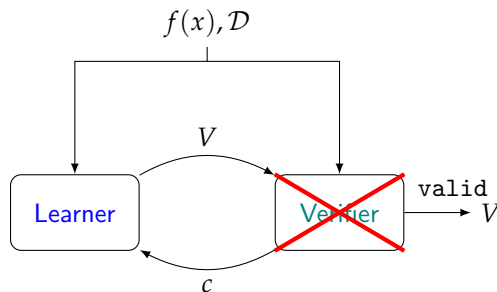
- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning



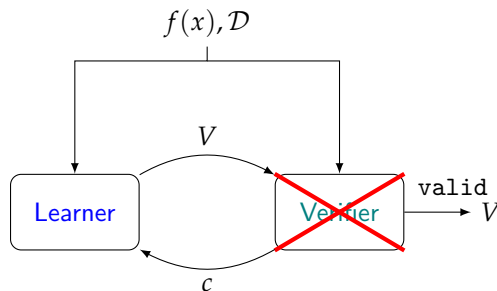
- earlier we relegated the use of SMT to the **Verifier** (**Prover**  $\rightarrow$  **Learner**)



- earlier we relegated the use of SMT to the **Verifier** (**Prover**  $\rightarrow$  **Learner**)



- earlier we relegated the use of SMT to the **Verifier** (**Prover**  $\rightarrow$  **Learner**)
- can we now eschew SMT solvers at all?
- from CEGIS to sample-driven synthesis, hence no need for models/SMT any longer



- model class:  $x^+ = f(x, w), x_0 \in X_I$
- behaviour: set of all **finite-horizon trajectories**  $\xi \in \Xi$
- consider **finite-horizon** specs  $\psi$  (e.g.,  **$T$ -step** reachability, safety, reach-avoid)
- ... now let's use some data, where now: **samples** =  **$T$ -step trajectories**

# PAC Learning - basics

- assume that a r.v.  $x \sim \Delta(\mathbb{R})$ , label
  - **Safe region** the interval  $[0, \infty)$
  - **Unsafe region** the interval  $(-\infty, 0)$
- given  $N$  i.i.d. samples  $\{x_i \geq 0\}_{i=1}^N$
- how likely is it that a new sample  $x < 0$ , and with what confidence?



*Low confidence*

# PAC Learning - basics

- assume that a r.v.  $x \sim \Delta(\mathbb{R})$ , label
  - **Safe region** the interval  $[0, \infty)$
  - **Unsafe region** the interval  $(-\infty, 0)$
- given  $N$  i.i.d. samples  $\{x_i \geq 0\}_{i=1}^N$
- how likely is it that a new sample  $x < 0$ , and with what confidence?



*High confidence*

# PAC Learning - basics

- assume that a r.v.  $x \sim \Delta(\mathbb{R})$ , label
  - **Safe region** the interval  $[0, \infty)$
  - **Unsafe region** the interval  $(-\infty, 0)$
- given  $N$  i.i.d. samples  $\{x_i \geq 0\}_{i=1}^N$
- how likely is it that a new sample  $x < 0$ , and with what confidence?
- we have that  $\mathbb{P}(x < 0) \leq \epsilon$   
 with confidence  $1 - \beta$ ,  
 where  $\beta = (1 - \epsilon)^N$



# PAC Learning - basics

- assume that a r.v.  $x \sim \Delta(\mathbb{R})$ , label
  - **Safe region** the interval  $[0, \infty)$
  - **Unsafe region** the interval  $(-\infty, 0)$
- given  $N$  i.i.d. samples  $\{x_i \geq 0\}_{i=1}^N$
- how likely is it that a new sample  $x < 0$ , and with what confidence?

- now having received  $k$  samples  $\{x_i < 0\}$  across  $\{x_i\}_{i=1}^N$ ,  
 then  $\mathbb{P}(x < 0) \leq \epsilon$  with confidence  $1 - \beta$ , where

$$\beta = \sum_{i=0}^k \binom{N}{i} \epsilon^i (1 - \epsilon)^{N-i}$$



- now apply this to *statistical property satisfaction*, where now **samples = trajectories**
- here samples  $\{x_i\}_{i=1}^N$  account whether property (e.g., **safety**) is satisfied, i.e.

$$x_i \geq 0 \Leftrightarrow \xi^i \models \psi$$

- hence, pick a  $\beta$ , take  $N$  **samples**, receiving  $k$ -many  $\{x_i < 0\}$ ,  
find  $\epsilon(k, \beta, N)$  from above, and conclude

$$\mathbb{P}^N \{ \{ \xi^i \}_{i=1}^N \in \Xi^N : \mathbb{P} \{ \xi \in \Xi : \xi \not\models \psi \} \leq \epsilon(k, \beta, N) \} \geq 1 - \beta$$

- now apply this to *statistical property satisfaction*, where now **samples = trajectories**
- here samples  $\{x_i\}_{i=1}^N$  account whether property (e.g., **safety**) is satisfied, i.e.

$$x_i \geq 0 \Leftrightarrow \xi^i \models \psi$$

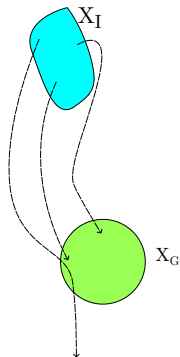
- hence, pick a  $\beta$ , take  $N$  **samples**, receiving  $k$ -many  $\{x_i < 0\}$ ,  
find  $\epsilon(k, \beta, N)$  from above, and conclude

$$\mathbb{P}^N \{ \{ \xi^i \}_{i=1}^N \in \Xi^N : \mathbb{P} \{ \xi \in \Xi : \xi \not\models \psi \} \leq \epsilon(k, \beta, N) \} \geq 1 - \beta$$

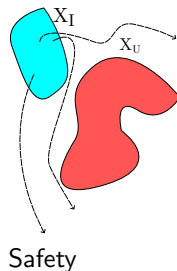
- next, we shall construct **certificates** from data and,
- leveraging PAC-based **scenario optimisation**,
- we shall provide **similar guarantees** (as formula above)

# Data-driven certificate synthesis, finite-sample guarantees

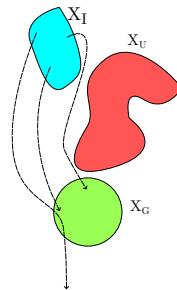
- model class:  $x^+ = f(x, w), x_0 \in X_I$
- behaviour: set of all finite-horizon trajectories  $\xi \in \Xi$
- consider finite-horizon specs  $\psi$ : e.g.,  **$T$ -step** reachability, safety, ...



Reachability



Safety



Reach-avoid

# Data-driven certificate synthesis, finite-sample guarantees

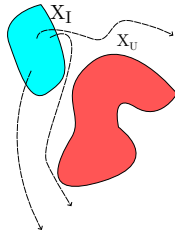
- model class:  $x^+ = f(x, w), x_0 \in X_I$
- behaviour: set of all finite-horizon trajectories  $\zeta \in \Xi$
- consider finite-horizon specs  $\psi$ : e.g.,  **$T$ -step** reachability, safety, ...
- proof rules tailored to finite-horizon specs  $\psi$
- e.g., sufficient conditions for  **$T$ -step** safety certificate:

$$V(x) \leq 0, \forall x \in X_I$$

$$V(x) > 0, \forall x \in X_U$$

$$V(x(k+1)) - V(x(k)) < \frac{1}{T} \left( \inf_{x \in X_U} V(x) - \sup_{x \in X_I} V(x) \right)$$

$$k = 0, \dots, T-1$$



# Data-driven certificate synthesis, finite-sample guarantees

- model class:  $x^+ = f(x, w), x_0 \in X_I$
- behaviour: set of all finite-horizon trajectories  $\xi \in \Xi$
- consider finite-horizon specs  $\psi$ : e.g.,  **$T$ -step** reachability, safety, ...
- proof rules tailored to finite-horizon specs  $\psi$
- e.g., sufficient conditions for  **$T$ -step** safety certificate:

$$V(x) \leq 0, \forall x \in X_I$$

$$V(x) > 0, \forall x \in X_U$$

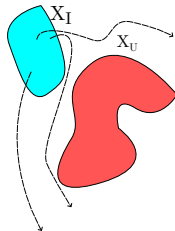
$$V(x(k+1)) - V(x(k)) < \frac{1}{T} \left( \inf_{x \in X_U} V(x) - \sup_{x \in X_I} V(x) \right)$$

$$k = 0, \dots, T-1$$

- data: consider  $N$  sampled  **$T$ -step** trajectories  $\{\xi^i\}_{i=1}^N \in \Xi^N$
- accordingly, sample-based learning of neural certificate  $V_\theta$
- loss function: **sample-dependent loss**  $l^s$  + **trajectory-dependent loss**  $l^\Delta$

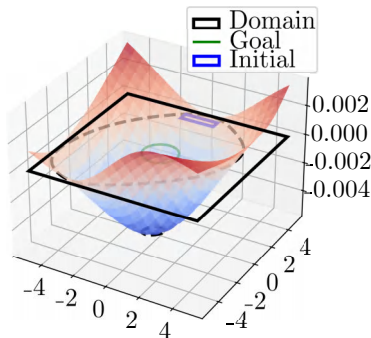
$$L(\theta, \xi) = l^s(\theta) + l^\Delta(\theta, \xi)$$

- obtain 'optimal'  $\theta^*$  via scenario optimisation

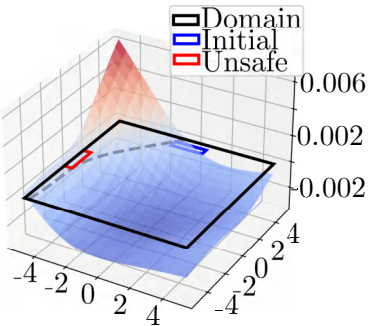


# Data-driven certificate synthesis, finite-sample guarantees

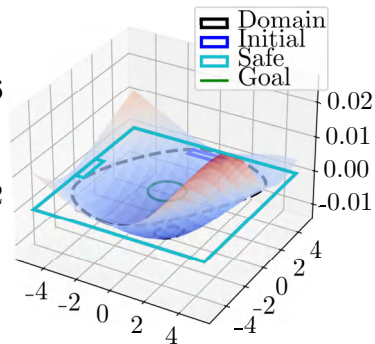
- model class:  $x^+ = f(x, w), x_0 \in X_I$
- behaviour: set of all finite-horizon trajectories  $\zeta \in \Xi$
- consider finite-horizon specs  $\psi$ : e.g.,  **$T$ -step** reachability, safety, ...
- proof rules tailored to finite-horizon specs  $\psi$
- data: consider  $N$  sampled  **$T$ -step** trajectories  $\{\zeta^i\}_{i=1}^N \in \Xi^N$
- accordingly, sample-based learning of neural certificate  $V_\theta$



Reachability



Safety/barrier



Reach-avoid

# Data-driven certificate synthesis, finite-sample guarantees

- model class:  $x^+ = f(x, w), x_0 \in X_I$
- behaviour: set of all finite-horizon trajectories  $\xi \in \Xi$
- consider finite-horizon specs  $\psi$ : e.g.,  **$T$ -step** reachability, safety, ...
- proof rules tailored to finite-horizon specs  $\psi$
- data: consider  $N$  sampled  **$T$ -step** trajectories  $\{\xi^i\}_{i=1}^N \in \Xi^N$
- accordingly, sample-based learning of neural certificate  $V_\theta$
- attain probabilistic property guarantee:  
 fix confidence level  $\beta \in (0, 1)$   
 find property violation level ("risk")  $\epsilon \in (0, 1)$ , such that

$$\mathbb{P}^N \left\{ \{\xi^i\}_{i=1}^N \in \Xi^N : \mathbb{P}\{\xi \in \Xi : \xi \not\models \psi(\xi)\} \leq \epsilon \right\} \geq 1 - \beta$$

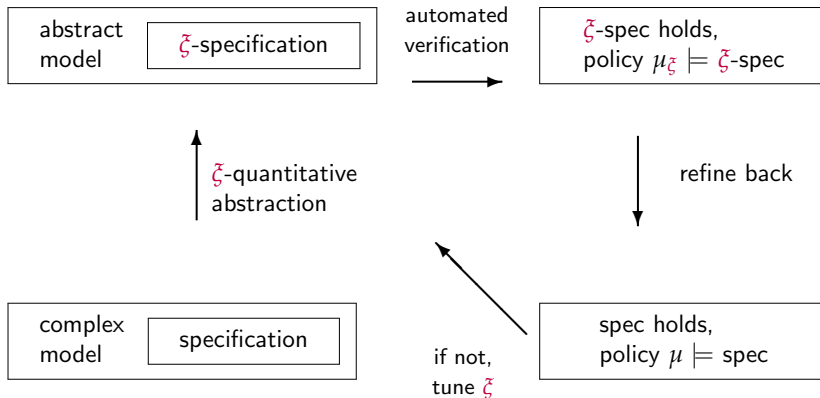
where  $\epsilon = \epsilon(R_N, \beta, N)$ ,

$R_N$  number of samples in  $\{\xi^i\}_{i=1}^N$  for which  $\psi(\xi^i)$  is violated  
 (namely, when 'optimal'  $V_{\theta^*}$  violates proof rules)

- 1 Context
- 2 *It's the Spec, stupid!*
- 3 Specification-driven Reinforcement Learning
- 4 Neural Proofs - Formal Synthesis of Neural Certificates
- 5 From Logic to Learning

# What has been left out of today's slides

- abstraction-based verification and synthesis
  - model-based
  - data-driven (connections with PAC, scenario, and RL)
  - CEGIS-based, cf. '*neural abstractions*' (above)



[cf. bib at end of deck]

Thank you for your attention

`oxcav.web.ox.ac.uk`

All images used are under Wikimedia CCAS license, or by author

## Selected References on Inductive (SMT and Neural) Synthesis of Certificates (and Controllers) and of Neural Abstractions

- A. Abate, D. Contro, M. Giacobbe, A. Martinez-Suñé, and D. Roy, "Value Functions as Supermartingale Certificates," SAIV26, In Press, 2026.
- N. Vertovec, F. B. Mathiesen, T. Badings, L. Laurenti and A. Abate, "Scalable Verification of Neural Control Barrier Functions Using Linear Bound Propagation," L4DC26
- A. Abate, M. Giacobbe, S. Ichtchenko, and D. Roy, "Complete  $\omega$ -Regular Supermartingale Certificates," arXiv:2605.21134, LICS26, LIPIcs, 2026.
- L. Rickard, A. Abate, K. Margellos, "Data-Driven Neural Certificate Synthesis," arXiv:2502.05510, Automatica, Vol 185, Mar 2026.
- A. Edwards, A. Peruffo and A. Abate, "A General Verification Framework for Dynamical and Control Models via Certificate Synthesis," arXiv:2309.06090, IEEE Annual Reviews in Control, Oct 2025.
- A. Edwards, A. Peruffo and A. Abate, "Fossil 2.0: Design, Usage and Impact of a Software Tool for Verification and Control of Dynamical Models," Science of Computer Programming, v. 247, Jan 2026.
- A. Abate, A. Edwards, M. Giacobbe, H. Punchihewa, and D. Roy, "Quantitative Verification with Neural Networks," Logical Methods in Computer Science, Volume 22, Issue 2, pp. 4:1–4:26, 2026.
- A. Abate, A. Edwards, M. Giacobbe, H. Punchihewa, and D. Roy, "Quantitative Neural Verification of Probabilistic Programs," CONCUR23, arXiv:2301.06136, 2023.
- A. Abate, M. Giacobbe, and D. Roy, "Quantitative Supermartingale Certificates," CAV25, LNCS 15934, pp. 3-28, 2025.
- A. Abate, M. Giacobbe, and D. Roy, "Quantitative Supermartingale Certificates," CAV25, LNCS 15934, pp. 3-28, 2025.
- A. Abate, M. Giacobbe, and D. Roy, "Stochastic Omega-Regular Verification and Control with Supermartingales," CAV24, LNCS 14683, pp. 395-419, 2024.
- A. Edwards, A. Peruffo and A. Abate, "Fossil 2.0: Formal Certificate Synthesis for the Verification and Control of Dynamical Models," HSCC24, ACM Press, vol. 26, pp 1-10, 2024. arXiv:2311.09793
- D. Roy, M. Giacobbe, and A. Abate, "Learning Probabilistic Termination Proofs," CAV21, LNCS 12760, pp. 3–26, 2021.
- A. Abate, D. Ahmed, A. Edwards, M. Giacobbe and A. Peruffo, "FOSSIL: A Software Tool for the Formal Synthesis of Lyapunov Functions and Barrier Certificates using Neural Networks," HSCC21, pp. 1-11, 2021.
- A. Abate, D. Ahmed and A. Peruffo, "Automated Formal Synthesis of Neural Barrier Certificates for Dynamical Models," TACAS21, LNCS 12651, pp. 370–388, 2021.
- D. Ahmed, A. Peruffo and A. Abate, "Automated and Sound Synthesis of Lyapunov Functions with SMT Solvers," TACAS20, LNCS 12078, pp. 97-114, 2020.
- A. Abate, D. Ahmed, M. Giacobbe and A. Peruffo "Automated Formal Synthesis of Lyapunov Neural Networks," IEEE Control Systems Letters, 5 (3), 773-778, 2020.
- A. Edwards, M. Giacobbe, and A. Abate, "On the Trade-off Between Efficiency and Precision of Neural Abstraction," QEST23, LNCS 14287, pp. 152-171, 2023.
- A. Abate, A. Edwards, and M. Giacobbe, "Neural Abstractions," NeurIPS22, Advances in Neural Information Processing Systems 35, 26432-26447, 2022.
- A. Abate, H. Barbosa, C. Barrett, C. David, P. Kesseli, D. Kroening, E. Polgreen, A. Reynolds, C. Tinelli, "Synthesising programs with non-trivial constants," Journal of Automated Reasoning (JAR), 67(2):19, 2023.
- A. Abate, I. Bessa, D. Cattaruzza, L. Cordeiro, C. David, P. Kesseli, D. Kroening and E. Polgreen, "Automated Formal Synthesis of Provably Safe Digital Controllers for Continuous Plants," Acta Informatica, 57(3), 2020.

## Selected Journal References on Model- and Sample-Based Formal Abstractions

- N. Vertovec, F.B. Mathiesen, F. Fabiano, L. Laurenti nad A. Abate, "Certified Neural Approximations of Nonlinear Dynamics," SAIV26, In Press, 2026.
- T. Badings, L. Romao, A. Abate, D. Parker, H. Poonawala, M. Stoelinga and N. Jansen, "Robust Control for Dynamical Systems with Non-Gaussian Noise via Formal Abstractions," JAIR, vol 76, pp.341-391, 2023.
- T.S. Badings, A. Abate, N. Jansen, D. Parker, H.A. Poonawala, and M. Stoelinga, "Sampling-Based Robust Control of Autonomous Systems with Non-Gaussian Noise," AAAI22, 36 (9), pp. 9669-9678, 2022.
- A. Lavaei, S. Soudjani, A. Abate, and M. Zamani, "Automated Verification and Synthesis of Stochastic Hybrid Systems: A Survey," Automatica, vol. 146, Dec. 2022.
- L. Laurenti, M. Lahijanian, A. Abate, L. Cardelli and M. Kwiatkowska, "Formal and Efficient Control Synthesis for Continuous-Time Stochastic Processes," IEEE Transactions on Automatic Control, vol. 66, no. 1, pp. 17-32, Jan 2021.
- S. Haesaert, S.E.Z. Soudjani, and A. Abate, "Verification of general Markov decision processes by approximate similarity relations and policy refinement," SIAM Journal on Control and Optimisation, vol. 55, nr. 4, pp. 2333-2367, 2017.
- I. Tkachev, A. Mereacre, J.-P. Katoen, and A. Abate, "Quantitative Model Checking of Controlled Discrete-Time Markov Processes," Information and Computation, vol. 253, nr. 1, pp. 1–35, 2017.
- S. Haesaert, N. Cauchi and A. Abate, "Certified policy synthesis for general Markov decision processes: An application in building automation systems," Performance Evaluation, vol. 117, pp. 75-103, 2017.
- S. Haesaert, at al., P.M.J. V.d. Hof, and A. Abate, "Data-driven and Model-based Verification via Bayesian Identification and Reachability Analysis," Automatica, vol. 79, pp. 115–126, 2017.
- S.E.Z. Soudjani and A. Abate, "Aggregation and Control of Populations of Thermostatically Controlled Loads by Formal Abstractions," IEEE Transactions on Control Systems Technology, vol. 23, nr. 3, pp. 975–990, 2015.
- S.E.Z. Soudjani and A. Abate, "Quantitative Approximation of the Probability Distribution of a Markov Process by Formal Abstractions," Logical Methods in Computer Science, Vol. 11, nr. 3, Oct. 2015.
- M. Zamani, P. Mohajerin Esfahani, R. Majumdar, A. Abate, and J. Lygeros, "Symbolic control of stochastic systems via approximately bisimilar finite abstractions," IEEE Transactions on Automatic Control, vol. 59 nr. 12, pp. 3135-3150, Dec. 2014.
- I. Tkachev and A. Abate, "Characterization and computation of infinite horizon specifications over Markov processes," Theoretical Computer Science, vol. 515, pp. 1-18, 2014.
- S. Soudjani and A. Abate, "Adaptive and Sequential Gridding for Abstraction and Verification of Stochastic Processes," SIAM Journal on Applied Dynamical Systems, vol. 12, nr. 2, pp. 921-956, 2013.
- A. Abate, J.P. Katoen, J. Lygeros and M. Prandini, "Approximate Model Checking of Stochastic Hybrid Systems," European Journal of Control, 16(6), 624-641, 2010.
- A. Abate, M. Prandini, J. Lygeros and S. Sastry, "Probabilistic Reachability and Safety Analysis of Controlled Discrete-Time Stochastic Hybrid Systems," Automatica, 44(11), 2724-2734, Nov. 2008.

## Selected references - Certified, Cautious, Multi-Objective and Meta Reinforcement Learning

- Y. Schnitzer, M. Jackermeier, A. Abate, and D. Parker, "Probabilistic Performance Guarantees for Multi-Task Reinforcement Learning," arXiv:2602.02098, ICML26, In Press, 2026.
- M. Jackermeier, M. Giuri, J. Cloete, and A. Abate, "Zero-Shot Instruction Following in RL via Structured LTL Representations," arXiv:2602.14344, 2026.
- J. Cloete, M. Jackermeier, I. Havoutis, and A. Abate, "PlatoLTL: Learning to Generalize Across Symbols in LTL Instructions for Multi-Task RL," arXiv:2601.22891, 2026.
- Y. Schnitzer, A. Abate, D. Parker, "Certifiably Robust Policies for Uncertain Parametric Environments," TACAS25, LNCS 15698, pp. 63-83, 2025.
- J. Cloete, N. Vertovec, and A. Abate, "SPoRt - Safe Policy Ratio: Certified Training and Deployment of Task Policies in Model-Free RL," IJCAI25, 2025.
- M. Jackermeier and A. Abate, "DeepLTL: Learning to Efficiently Satisfy Complex LTL Instructions," ICLR25, 2025.
- M. Hasanbeig, A. Abate and D. Kroening, "Certified Reinforcement Learning with Logic Guidance," arXiv:1801.08099, Artificial Intelligence (AIJ), vol 322, p. 103949, 2023.
- R. Mitta, H. Hasanbeig, Jun W, D. Kroening, Y. Kantaros, and A. Abate, "Safeguarded Progress in Reinforcement Learning: Safe Bayesian Exploration for Control Policy Synthesis," AAAI24, 2024.
- A. Abate, Y. Almula, J. Fox, D. Hyland, and M. Wooldridge, "Learning Task Automata for RL Using Hidden Markov Models," ECAI23, 2023.
- M. Hasanbeig, A. Abate, and D. Kroening, "Logically-Constrained Neural Fitted Q-Iteration," AAMAS19, pp. 2012-2014, 2019.
- M. Hasanbeig, Y. Kantaros, A. Abate, D. Kroening, G. Pappas and I. Lee, "Reinforcement Learning for Temporal Logic Control Synthesis with Probabilistic Satisfaction Guarantees," CDC19, pp. 5338-5343, 2019.
- M. Hasanbeig, A. Abate and D. Kroening, "Cautious Reinforcement Learning with Logical Constraints," AAMAS20, pp. 483-491, 2020.
- M. Hasanbeig, D. Kroening and A. Abate, "Deep Reinforcement Learning with Temporal Logics," FORMATS20, LNCS 12288, pp. 1-22, 2020.
- M. Hasanbeig, N. Jeppu, A. Abate, T. Melham and D. Kroening, "DeepSynth: Program Synthesis for Automatic Task Segmentation in Deep Reinforcement Learning," AAAI 2021.
- L. Hammond, A. Abate, J. Gutierrez, and M. Wooldridge, "Multi-Agent Reinforcement Learning with Temporal Logic Specifications," AAAMAS 2021.
- J. Skalse, L. Hammond, and A. Abate, "Lexicographic Multi-Objective Reinforcement Learning," IJCAI-ECAI 2022.
- J. Skalse, L. Farnik, S. Motwani, E. Jenner, A. Gleave, and A. Abate, "STARC: A General Framework For Quantifying Differences Between Reward Functions," ICLR24, In Print, 2023.
- J. Skalse, M. Farrugia-Roberts, S. Russell, A. Abate, and A. Gleave, "Invariance in Policy Optimisation and Partial Identifiability in Reward Learning," ICML23, PMLR, pp. 32033-32058, 2023.
- J. Skalse and A. Abate, "Misspecification in Inverse Reinforcement Learning," AAAI23, vol. 37, nr. 12, pp. 15136-15143, 2023.

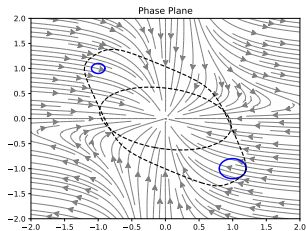


| Benchmark   | CEGIS (this work) |        |         |       | BC <sup>1</sup> |        |         | SOS <sup>2</sup> |        |
|-------------|-------------------|--------|---------|-------|-----------------|--------|---------|------------------|--------|
|             | Learn             | Verify | Samples | Iters | Learn           | Verify | Samples | Synth            | Verify |
| Darboux     | 31.6              | 0.01   | 0.5 k   | 2     | 54.9            | 20.8   | 65 k    | ×                | —      |
| Exponential | 15.9              | 0.07   | 1.5 k   | 2     | 234.0           | 11.3   | 65 k    | ×                | —      |
| Obstacle    | 55.5              | 1.83   | 2.0 k   | 9     | 3165.3          | 1003.3 | 2097 k  | ×                | —      |
| Polynomial  | 64.5              | 4.20   | 2.3 k   | 2     | 1731.0          | 635.3  | 65 k    | 8.10             | ×      |
| Hybrid mod  | 0.58              | 2.01   | 0.5 k   | 1     | —               | —      | —       | 12.30            | 0.11   |
| 4-d ODE     | 29.31             | 0.07   | 1 k     | 1     | —               | —      | —       | 12.90            | OOT    |
| 6-d ODE     | 89.52             | 1.61   | 1 k     | 3     | —               | —      | —       | 16.60            | OOT    |
| 8-d ODE     | 104.5             | 82.51  | 1 k     | 3     | —               | —      | —       | 26.10            | OOT    |

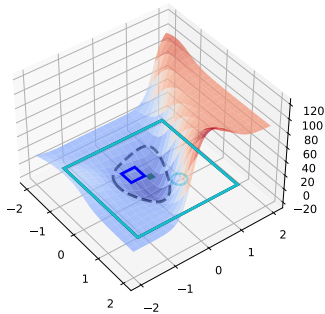
- time for Learning and Verification steps in [sec]
- ‘Samples’ = size of input data for Learner (in thousands)
- ‘Iters’ = number of iterations of CEGIS loop
- × = synthesis or verification failure, OOT = verification timeout

<sup>1</sup> H. Zhao, X. Zeng, T. Chen, and Z. Liu. Synthesizing Barrier Certificates Using Neural Networks. In Proceedings of the 23rd International Conference on Hybrid Systems: Computation and Control, HSCC, 2020.

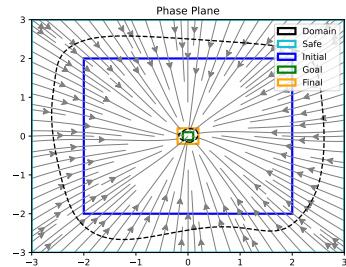
<sup>2</sup> A. Papachristodoulou, J. Anderson, G. Valmorbida, S. Prajna, P. Seiler, and P. A. Parrilo. SOSTOOLS: Sum of squares optimization toolbox for MATLAB, 2013.



ROA for NL model,  
 non-poly Lyapunov,  
 2 disjoint initial sets



RWA: reach-while-avoid



RAR certificate for closed-loop  
 NL model

dashed lines: level sets; dark blue:  $\mathcal{I}$ ; light blue:  $\mathcal{S}$ ; green:  $\mathcal{G}$ ; orange:  $\mathcal{F}$



so far we have seen that:

- ranking supermartingales for almost-sure reachability
- almost sure persistence, or a.s. recurrence
- Streett supermartingales for almost-sure reactivity conditions  
and indeed **many other similar proof rules** ...

so far we have seen that:

- ranking supermartingales for almost-sure reachability
- almost sure persistence, or a.s. recurrence
- Streett supermartingales for almost-sure reactivity conditions  
and indeed **many other similar proof rules** ...
- restrict state space to **supporting invariant**  $I \subseteq \text{State}$ , so that:

$$\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$$

so far we have seen that:

- ranking supermartingales for almost-sure reachability
- almost sure persistence, or a.s. recurrence
- Streett supermartingales for almost-sure reactivity conditions  
and indeed **many other similar proof rules** ...
- restrict state space to **supporting invariant**  $I \subseteq \text{State}$ , so that:

$$\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$$

- now, we can generalise any **almost-sure proof rule** to **quantitative** supermartingale certificates:

$$\mathbb{P}(\text{Specification}) \geq p$$

- this is done via a “decomposition” result (next)

- soundness of decomposition [AA et al, CAV25] - suppose:
  - ①  $\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$
  - ②  $\mathbb{P}(\mathbf{G} I) \geq p$
- then  $\mathbb{P}(\text{Specification}) \geq p$

- soundness of decomposition [AA et al, CAV25] - suppose:
  - ①  $\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$
  - ②  $\mathbb{P}(\mathbf{G} I) \geq p$
- then  $\mathbb{P}(\text{Specification}) \geq p$
  
- special case:
  - ① if  $\mathbb{P}(\text{Streett acceptance} \mid \mathbf{G} I) = 1$   
(e.g., via Streett supermartingale & supporting invariant, [AA et. al., CAV'24])
  - ② if  $\mathbb{P}(\mathbf{G} I) \geq p$   
(e.g., via stochastic invariant (supermartingale) [Kushner, 1965] or repulsing supermartingales [Chatterjee et.al., POPL'17][Takisaka et.a., ATVA'18])
- then  $\mathbb{P}(\text{Streett acceptance}) \geq p$

• **question:** does there always exist a region  $I \subseteq \text{State}$  such that:

- ①  $\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1,$
- ②  $\mathbb{P}(\mathbf{G} I) = \mathbb{P}(\text{Specification})$

---

<sup>3</sup>A language is invariant under addition/deletion of any finite prefix to an infinite trajectory

- **question:** does there always exist a region  $I \subseteq \text{State}$  such that:
  - ①  $\mathbb{P}(\text{Specification} \mid \mathbf{G} I) = 1$ ,
  - ②  $\mathbb{P}(\mathbf{G} I) = \mathbb{P}(\text{Specification})$
- **poositive answer:** suppose  $L$  is a **shift-invariant** specification<sup>3</sup>, then the following holds:

## Theorem ( $\varepsilon$ -completeness for gMC [AA et al, CAV25] )

*For every  $\varepsilon > 0$ , there exists a region  $I$  such that*

- ①  $\mathbb{P}(L \mid \mathbf{G} I) = 1$
- ②  $\mathbb{P}(\mathbf{G} I) \geq \mathbb{P}(L) - \varepsilon$

## Theorem (Completeness for finite MC [AA et al, CAV25] )

*There exists a region  $I$  such that*

- ①  $\mathbb{P}(L \mid \mathbf{G} I) = 1$
- ②  $\mathbb{P}(\mathbf{G} I) = \mathbb{P}(L)$

<sup>3</sup>A language is invariant under addition/deletion of any finite prefix to an infinite trajectory

## Theorem

Let  $(A_1, B_1) \in \Sigma^2, \dots, (A_k, B_k) \in \Sigma^2$  be a Streett acceptance condition.  
Suppose that there exists a measurable set  $I \in \Sigma$  and  $k+1$  measurable functions  $V_0, \dots, V_k: \text{State} \rightarrow \mathbb{R}_{\geq 0}$  such that the following conditions hold:

$$\forall s \in I: PV_0(s) \leq V_0(s),$$

$$\forall s \notin I: V_0(s) \geq 1,$$

$$\forall s \in I \cap (A_i \setminus B_i): PV_i(s) \leq V_i(s) - \varepsilon \quad \text{for } i = 1, \dots, k,$$

$$\forall s \in I \cap B_i: PV_i(s) \leq V_i(s) + M \quad \text{for } i = 1, \dots, k,$$

$$\forall s \in I \setminus (A_i \cup B_i): PV_i(s) \leq V_i(s) \quad \text{for } i = 1, \dots, k,$$

for some constants  $\varepsilon, M > 0$ . Then,

$$\mathbb{P}_\mu \left( \bigwedge_{i=1}^k \sum_{n=0}^{\infty} \mathbf{1}_{A_i}(\Phi_n) < \infty \vee \sum_{n=0}^{\infty} \mathbf{1}_{B_i}(\Phi_n) = \infty \right) \geq 1 - \mu V_0$$



# From dynamical to *switched* models

A **discrete-time (arbitrarily-)switched model** is

$$x(k+1) = f_{\sigma(k)}(x(k)), \quad \sigma(k) \in \{1, \dots, M\}$$

here **switching signal**  $\sigma : \mathbb{N} \rightarrow \{1, \dots, M\}$  determines sequence of modes (and of dynamics). Switching is **arbitrary**, *i.e.*, every sequence in  $\Sigma^{\mathbb{N}}$  is possible.

## Example

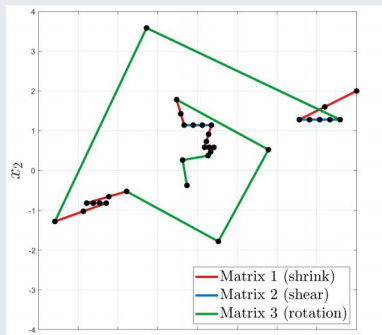
Consider linear case  $f_{\sigma(k)}(x(k)) = A_{\sigma(k)}x(k)$  and

$$A_1 = \begin{bmatrix} 0.8 & 0 \\ 0 & 0.8 \end{bmatrix}, A_2 = \begin{bmatrix} 1.0 & 0.2 \\ 0 & 1.0 \end{bmatrix}, A_3 = \begin{bmatrix} 0 & -1.0 \\ 1.0 & 0 \end{bmatrix}$$

$A_1$  shrinks,  $A_2$  shears,  $A_3$  rotates.

Switching sequence:

$$\sigma = 11222233112221133311222111221133 \dots$$



# From dynamical to *switched* models

A **discrete-time (arbitrarily-)switched model** is

$$x(k+1) = f_{\sigma_k}(x(k)), \quad \sigma(k) \in \{1, \dots, M\}$$

here **switching signal**  $\sigma : \mathbb{N} \rightarrow \{1, \dots, M\}$  determines sequence of modes (and of dynamics).  
Switching is **arbitrary**, *i.e.*, every sequence in  $\Sigma^{\mathbb{N}}$  is possible.

## Example

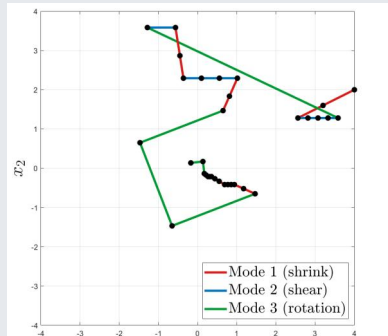
Consider linear case  $f_{\sigma(k)}(x(k)) = A_{\sigma(k)}x(k)$  and

$$A_1 = \begin{bmatrix} 0.8 & 0 \\ 0 & 0.8 \end{bmatrix}, A_2 = \begin{bmatrix} 1.0 & 0.2 \\ 0 & 1.0 \end{bmatrix}, A_3 = \begin{bmatrix} 0 & -1.0 \\ 1.0 & 0 \end{bmatrix}$$

$A_1$  shrinks,  $A_2$  shears,  $A_3$  rotates.

Switching sequence:

$$\sigma = 1122223\textcolor{red}{2}112221133311222111221133\dots$$



A **discrete-time (arbitrarily-)switched model** is

$$x(k+1) = f_{\sigma_k}(x(k)), \quad \sigma(k) \in \{1, \dots, M\}$$

here **switching signal**  $\sigma : \mathbb{N} \rightarrow \{1, \dots, M\}$  determines sequence of modes (and of dynamics). Switching is **arbitrary**, *i.e.*, every sequence in  $\Sigma^{\mathbb{N}}$  is possible.

## Common stability certificate

A **common Lyapunov function** is a single function  $V : \mathbb{R}^n \rightarrow \mathbb{R}_{\geq 0}$  with one inequality constraint per mode:

$$\forall m \in \{1, \dots, M\}, \forall x \in \mathbb{R}^n : V(f_m(x)) \leq V(x)$$

Can be **conservative** for specific templates

single function  $\rightarrow$  finite collection of Lyapunov pieces, coupled by **graph**

## Path-complete Lyapunov function

Given a directed and labelled graph  $\mathcal{G} = (S, E)$ ,

a **path-complete Lyapunov function** is a finite collection  $\{V_s : s \in S\}$  s.t.

$$\forall e = (s, m, d) \in E, \forall x \in \mathbb{R}^n : V_d(f_m(x)) \leq V_s(x).$$

**Key idea:** **path-complete graph**  $\rightarrow$  every finite word is a path

local inequalities are chained along any switching sequence

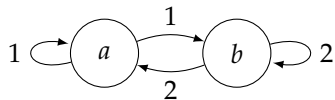
$\rightarrow$  stability is guaranteed for arbitrary switching

$$\begin{array}{ccccccc} s_0 & \xrightarrow{\sigma_0} & s_1 & \xrightarrow{\sigma_1} & s_2 & \xrightarrow{\sigma_2} \cdots & s_K \\ V_{s_0}(x_0) & \geq & V_{s_1}(x_1) & \geq & V_{s_2}(x_2) & \geq \cdots & V_{s_K}(x_K) \end{array}$$

# Path-complete framework:

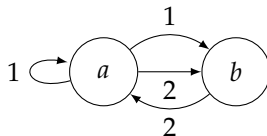
different *graphs* yield different proof rules

graph 1



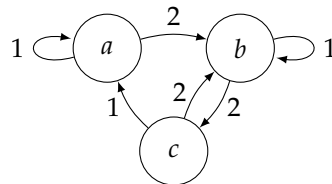
$$\begin{aligned} V_a(f_1(x)) &\leq V_a(x) \\ V_b(f_1(x)) &\leq V_a(x) \\ V_b(f_2(x)) &\leq V_b(x) \\ V_a(f_2(x)) &\leq V_b(x) \end{aligned}$$

graph 2



$$\begin{aligned} V_a(f_1(x)) &\leq V_a(x) \\ V_b(f_1(x)) &\leq V_a(x) \\ V_b(f_2(x)) &\leq V_a(x) \\ V_a(f_2(x)) &\leq V_b(x) \end{aligned}$$

graph 3



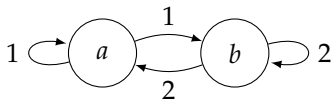
$$\begin{aligned} V_a(f_1(x)) &\leq V_a(x) & V_c(f_2(x)) &\leq V_b(x) \\ V_b(f_2(x)) &\leq V_a(x) & V_a(f_1(x)) &\leq V_c(x) \\ V_b(f_1(x)) &\leq V_b(x) & V_b(f_2(x)) &\leq V_c(x) \end{aligned}$$

graph structure is combinatorial, certificate inequalities are algebraic  
→ SMT solvers as a natural tool for synthesis

# Path-complete framework:

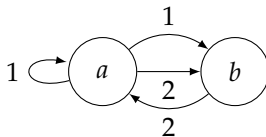
different *graphs* yield different proof rules

graph 1



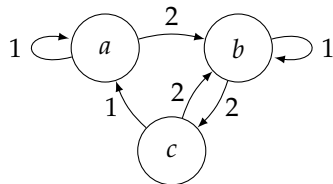
$$\begin{aligned} V_a(f_1(x)) &\leq V_a(x) \\ V_b(f_1(x)) &\leq V_a(x) \\ V_b(f_2(x)) &\leq V_b(x) \\ V_a(f_2(x)) &\leq V_b(x) \end{aligned}$$

graph 2



$$\begin{aligned} V_a(f_1(x)) &\leq V_a(x) \\ V_b(f_1(x)) &\leq V_a(x) \\ V_b(f_2(x)) &\leq V_a(x) \\ V_a(f_2(x)) &\leq V_b(x) \end{aligned}$$

graph 3



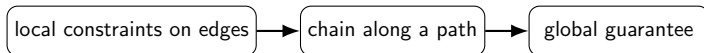
$$\begin{aligned} V_a(f_1(x)) &\leq V_a(x) & V_c(f_2(x)) &\leq V_b(x) \\ V_b(f_2(x)) &\leq V_a(x) & V_a(f_1(x)) &\leq V_c(x) \\ V_b(f_1(x)) &\leq V_b(x) & V_b(f_2(x)) &\leq V_c(x) \end{aligned}$$

## Current work - *beyond stability*

Path-complete ranking supermartingales for stochastic switched systems for termination+safety (reach-avoid), and beyond

## Same framework, different certificates

Graphs still encode how quantity locally decreases, while path-completeness propagates it along every path



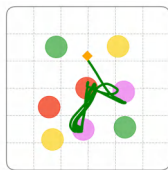
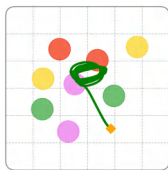
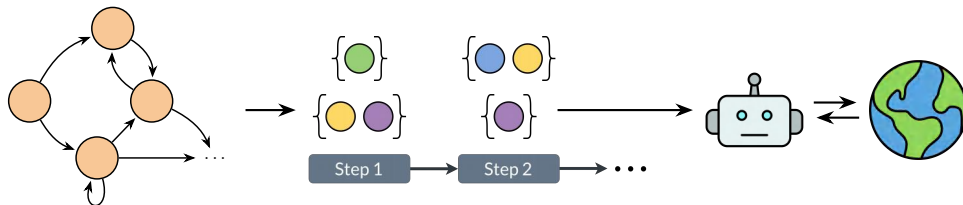
## Current work

Path-complete ranking supermartingales for stochastic switched systems for termination+safety (reach-avoid), and beyond

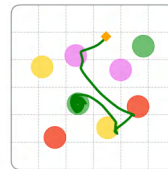
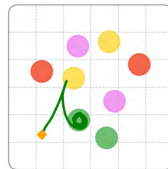


# DeepLTL: zero-shot tasks generalisation

progress through automaton states as **reach-avoid** sequences



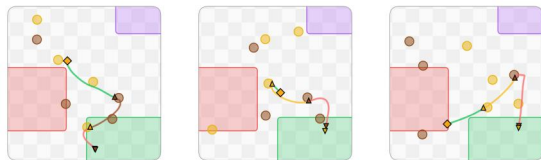
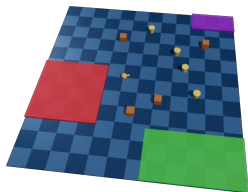
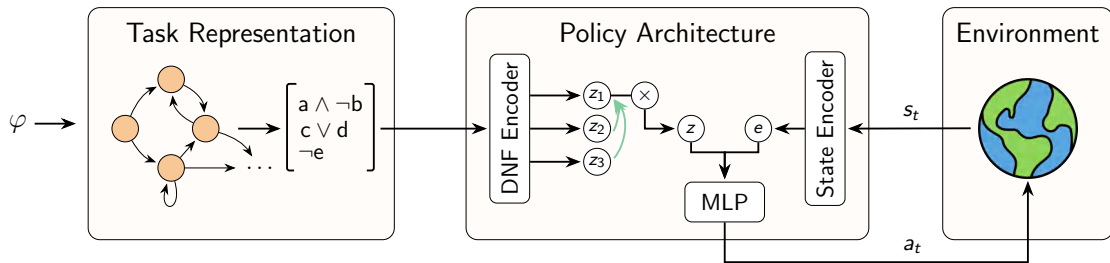
$F(\bullet \text{ red}) \wedge FG(\bullet \text{ purple})$



$F(\bullet \text{ yellow}) \wedge FG(\bullet \text{ green}) \wedge G(\bullet \text{ purple} \rightarrow F(\bullet \text{ red}))$

# StructLTL: learning structured task representations

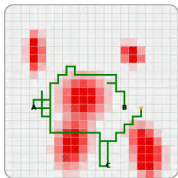
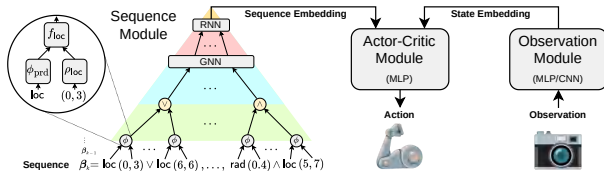
improved generalisation via **structured** learned task embeddings



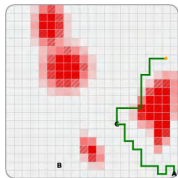
$$F(\text{vase} \wedge \text{crate} \wedge \bullet \text{green} \wedge (\bullet \text{green} \cup (\neg \text{vase} \wedge \neg \text{crate} \wedge \bullet \text{green})))$$

# PlatoLTL: parametric generalisation across symbols

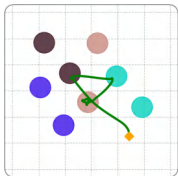
model propositions as parameterised instances of **atomic predicates**



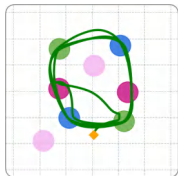
$$F(\text{loc}(A) \wedge (\neg \text{rad}(0.8) U \text{loc}(B))) \wedge F \text{loc}(B)$$



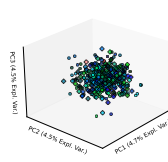
$$(\text{rad}(0.4) \Rightarrow (\neg \text{loc}(A) U (\text{loc}(B) \vee \text{loc}(C)))) U \text{loc}(A)$$



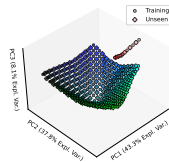
$$F G \text{at}(\bullet) \wedge F(\text{at}(\bullet) \wedge F \text{at}(\bullet))$$



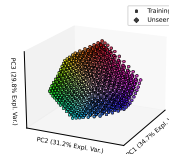
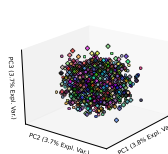
$$G F \text{at}(\bullet) \wedge G F \text{at}(\bullet) \wedge G F \text{at}(\bullet) \wedge G \neg \text{at}(\bullet)$$



DeepLTL



PlatoLTL



proposition embeddings

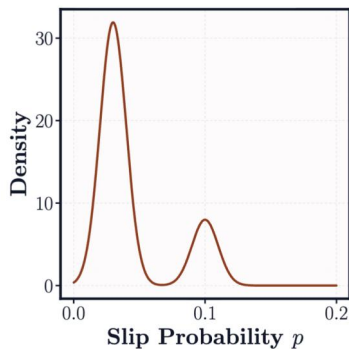
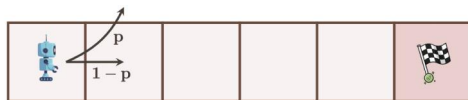


- RL in single MDP/‘task’  $\mathcal{M}$ :

$$J_{\mathcal{M}}(\pi) = \begin{cases} \mathbb{E}_{\tau \sim \mathcal{M}^{\pi}} \left[ \sum_{t=0}^T \gamma^t r_t \right] \\ \mathbb{P}_{\tau \sim \mathcal{M}^{\pi}} [\tau \models \psi] \end{cases}$$

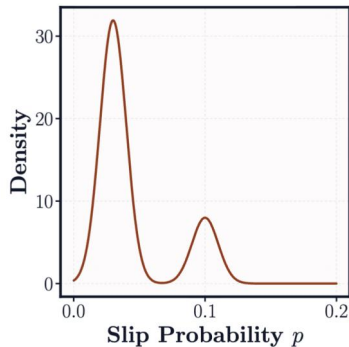
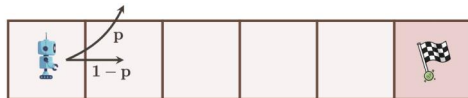
- meta-RL over unknown distribution  $\mathcal{D}$ :

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\mathcal{M} \sim \mathcal{D}} [J_{\mathcal{M}}(\pi)]$$



# Certifying multi-task/environment Reinforcement Learning

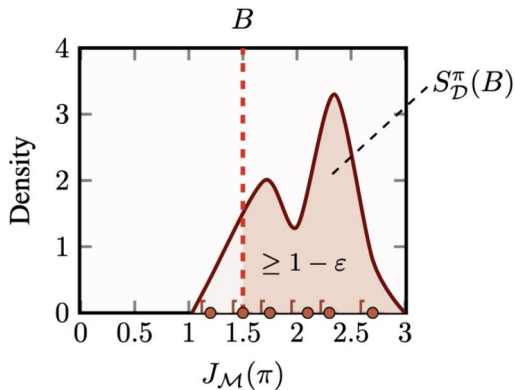
- **given:** single policy  $\pi$  that performs/generalises well across ‘tasks’
- ‘tasks’ encompass different dynamics, initial states, constraints, rewards/objectives
- $\mathcal{D}$  is arbitrary and unknown
- **goal:** certify zero-shot performance of  $\pi$  on new, unseen task  $\mathcal{M} \sim \mathcal{D}$



- fix ‘performance’ threshold  $B \in \mathbb{R}$ ,  
accordingly define **safety**:

$$S_D^\pi(B) = \Pr_{\mathcal{M} \sim \mathcal{D}} [J_{\mathcal{M}}(\pi) \geq B]$$

- safety is probability for policy  $\pi$  to achieve at least performance  $B$  on new, unseen task  $\mathcal{M} \sim \mathcal{D}$
- risk of falling below  $B$  is then  $1 - S_D^\pi(B)$



- given a trained policy  $\pi$ :

- sample tasks

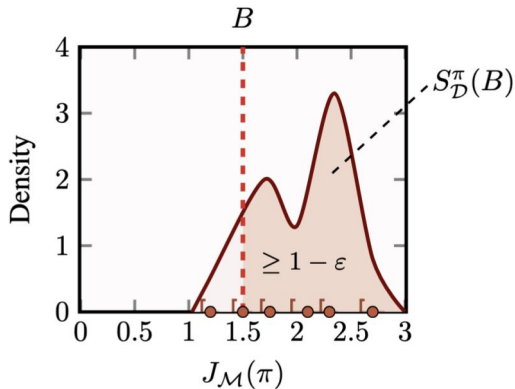
$$\mathbb{M} = \{\mathcal{M}_i \sim \mathcal{D}\}_{i=1}^n$$

- collect rollouts

$$\mathbb{T}_i = \{\tau_{i,j} \sim \mathcal{M}_i^\pi\}_{j=1}^{m_i}$$

- **two uncertainty layers:**

- finite task samples from an unknown  $\mathcal{D}$
- finite rollouts within each sampled task



- derive from finite data a **PAC** statement of the form:

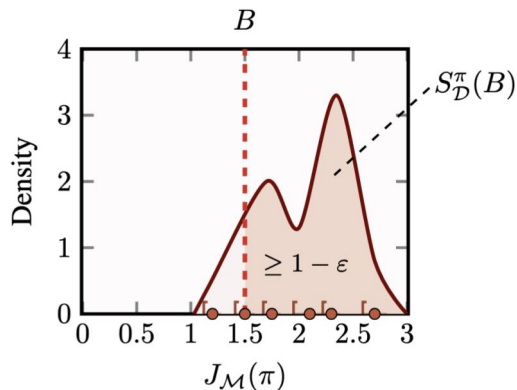
$$\Pr[S_D^\pi(B) \geq 1 - \varepsilon] \geq 1 - \beta,$$

for user-chosen  $\beta \in (0,1)$

and derived (as large as possible)  $1 - \varepsilon$

by combining **two uncertainty layers**,

via two steps (next)



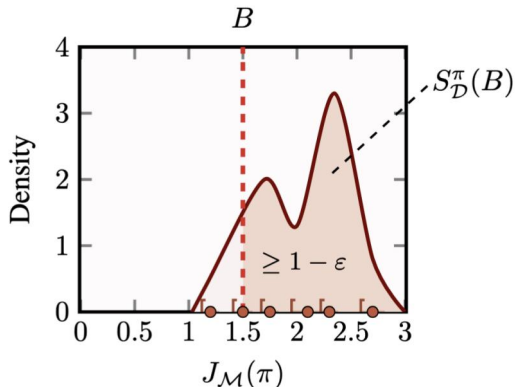
# Step 1: Per-Task Lower Confidence Bounds

- for single task  $\mathcal{M}_i$ ,  
construct lower confidence bound  $\tilde{J}_i^\delta$  with

$$\Pr \left[ J_{\mathcal{M}_i}(\pi) \geq \tilde{J}_i^\delta \right] \geq 1 - \delta,$$

from collected rollouts  $\mathbb{T}_i$  - can use:

- binary metrics (specs): Clopper–Pearson
- reward metrics: Hoeffding, DKW, empirical Bernstein
- $\delta$  (user-chosen) controls per-task coverage



## Step 2: From Tasks to Distribution-Level Guarantees

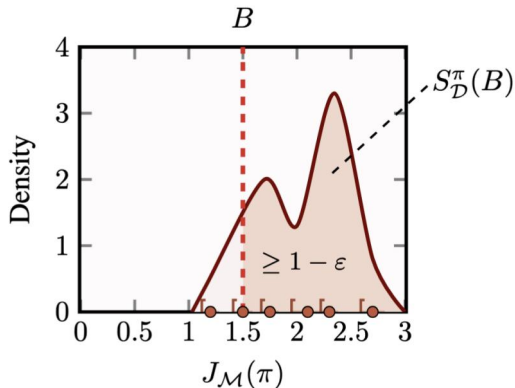
- for threshold  $B$ , count ‘violations’ (as before)

$$k(B) = \left| \{i : \tilde{f}_i^\delta < B\} \right|$$

- then, given  $\beta$ ,  
compute  $\varepsilon$  (eqn’s similar as above), s.t.

$$\Pr[S_{\mathcal{D}}^{\pi}(B) \geq 1 - \varepsilon] \geq 1 - \beta$$

- this
  - handles unknown task distribution  $\mathcal{D}$
  - combines **two** layers of uncertainty



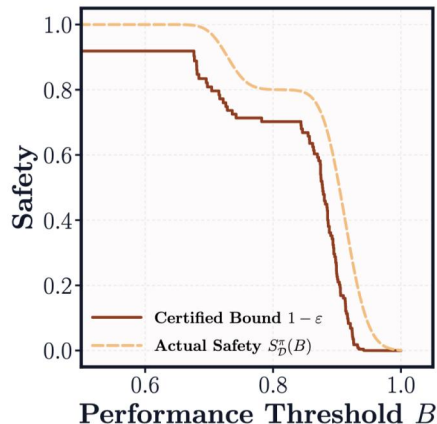
# Guarantees on safety depend on tasks vs. rollouts

- returned  $\varepsilon$  is such that

$$\Pr[S_D^\pi(B) \geq 1 - \varepsilon] \geq 1 - \beta$$

- features:

- more rollouts tighten  $\tilde{f}_i^\delta$
- more tasks sharpen task-level generalisation
- $\beta$  - global confidence budget



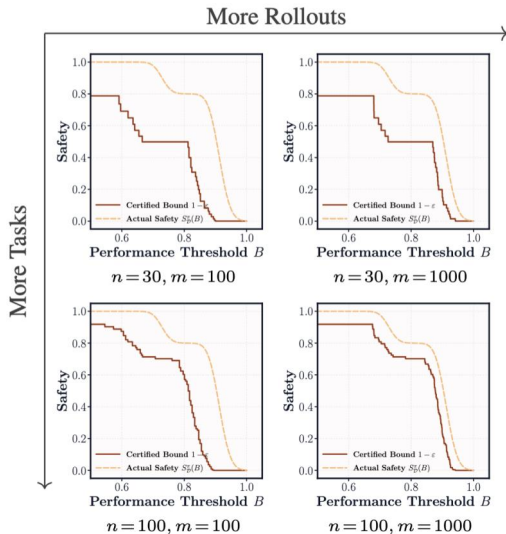
# Guarantees on safety depend on tasks vs. rollouts

- returned  $\varepsilon$  is such that

$$\Pr [S_D^\pi(B) \geq 1 - \varepsilon] \geq 1 - \beta$$

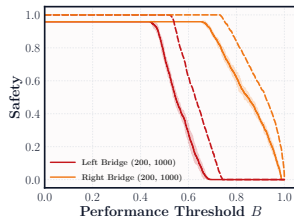
- features:

- more rollouts tighten  $\tilde{f}_i^\delta$
- more tasks sharpen task-level generalisation
- $\beta$  - global confidence budget

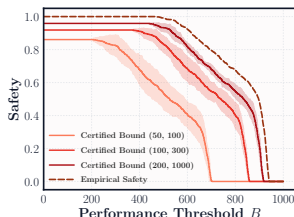


# Experiments

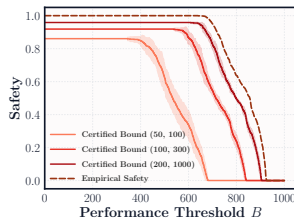
- guarantees (bounds) tightly track empirical safety
- bounds practically informative for realistic sample sizes (tasks  $n$  and rollouts  $m$ )
- no dependence on state-space dimension or domain complexity
- independent of task or specification structure



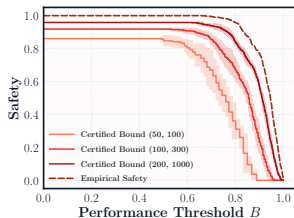
Bridge World



Cheetah



Walker



Zones